Practical Project

Nedelcu Radu

Iuga Paula

Group 332

FMI UNIBUC

3rd Year, 1st Semester

ORG Link

Contents

Practical project **1**

Business case 3

SF Cloud 4

Data Model 5

Record types and Page layouts 11

Security model 15

Data quality 19

Process Automation 23

Custom code automations 29

Integration 31

Lightning Web Components 32

Branded UI 33

Reports & Dasboards: 34

Tests 36

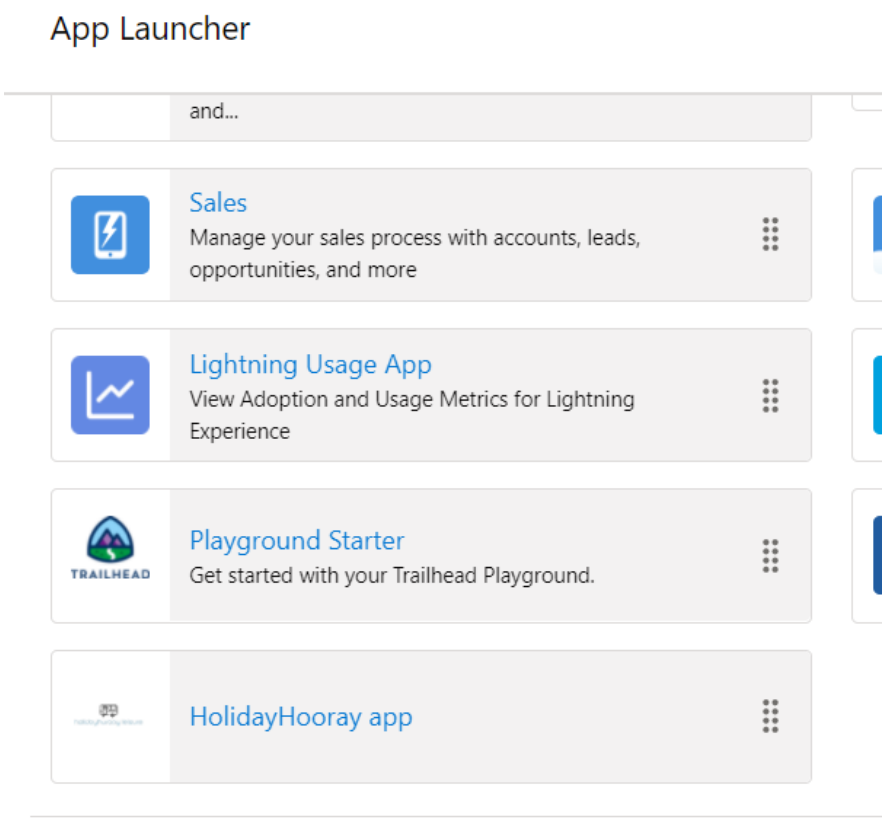
Business Case

holidayhuraay leisure

HolidayHuraay Leisure is a leisure & hospitality company that offers holiday home ownership and caravan holidays across beautiful areas from Europe. They make each vacation one to remember by offering their clientele lots of fun and relaxing activities.

The main point of their business is getting customer service right so they can make their clients happy and grow the business. They want to use Salesforce in order to give their regulars suitable advice on choosing their holiday escapes and to respond quickly to their questions. Moreover, they want to organize all of their team in specialized groups to solve each customer's problem.

They would like to get all the needed information about a client fast and use tools to measure and analyze the business so that they can attract more future buyers.



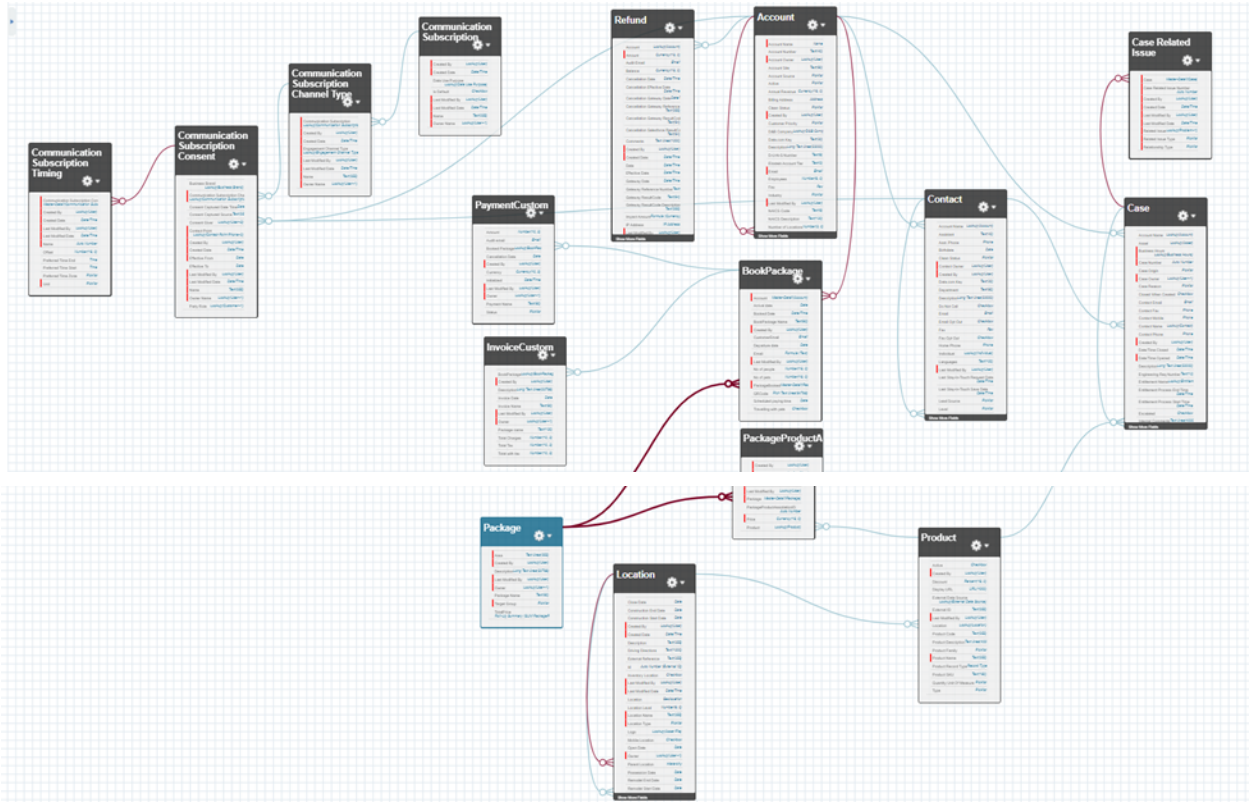
HoolidayHooray Lighting App

SF Cloud

The company should use the Service Cloud because it is a CRM platform for customer service and support. With Service cloud, they can offer support everywhere and send immediate response messages. There, they can track a client's issue using cases. Moreover, they can get many people working together to solve a case using Case Teams. In addition to this, they can track what work has been done recently, what are the most common customer problems and can respond to users' emails/messages from different apps using only one platform.

Service Cloud also helps them to automatise the process of cases and to keep all the accounts centralized and organized in a single place, so finding information about a buyer becomes fast and easy.

Data Model

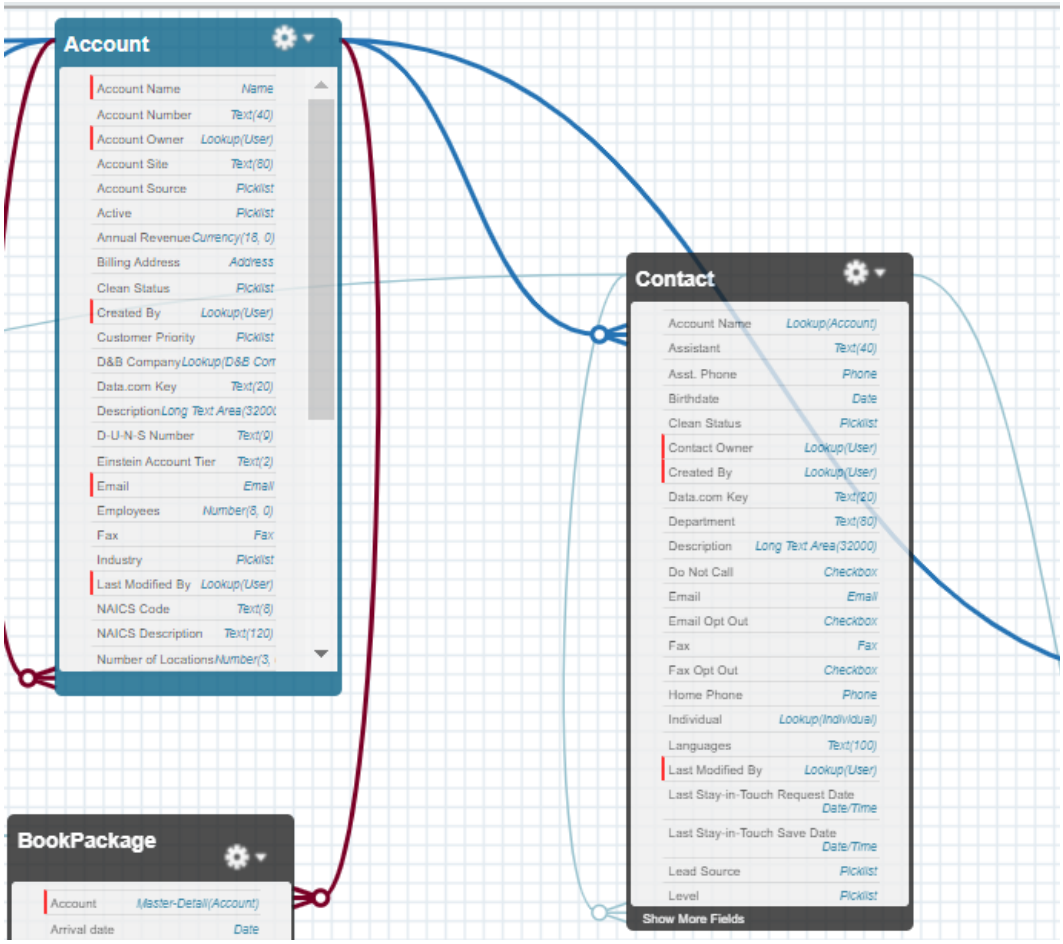


- **Accounts & Contacts**

  Accounts represent companies that want to collaborate with HolidayHuraay Leisure (e.g. Booking, HomeAway, travelling agencies like smartTours and TuiTravel). ***Contacts*** are the employees from these companies. Thus, the relationship between ***accounts*** and ***contacts*** represents the relationship between companies and the people that work for them.

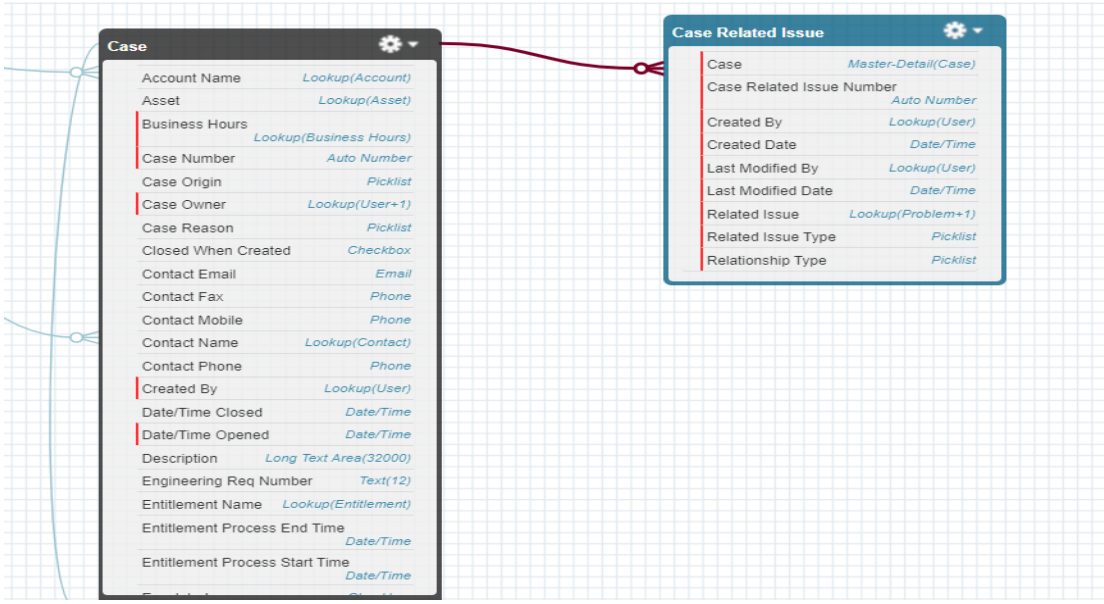
  Besides a company, an ***account*** can also represent a normal customer that uses HolidayHuraay Leisure services. The relationship ***account-account*** represents the relationship between customers and other customers e.g. ChristianTour and TravelTour (the first one takes clients from the second one to form a group of tourists for a destination) or a happy customer – Billy Jones - and a friend of his – Anne Antoine – that was brought at HolidayHuraay Leisure by Billy – so he has some extra sales for introducing other people to the company's services.

  The relationship between **contact and contact** is the relationship between an employee from a customer company and his manager or boss.



- **Cases**

  Cases track customers issues regarding a product or questions.
  A case can be divided into subcases which are stored in ***Case Related Issue*** object e.g. a delayed trip might link to difficulties in changing accommodations.

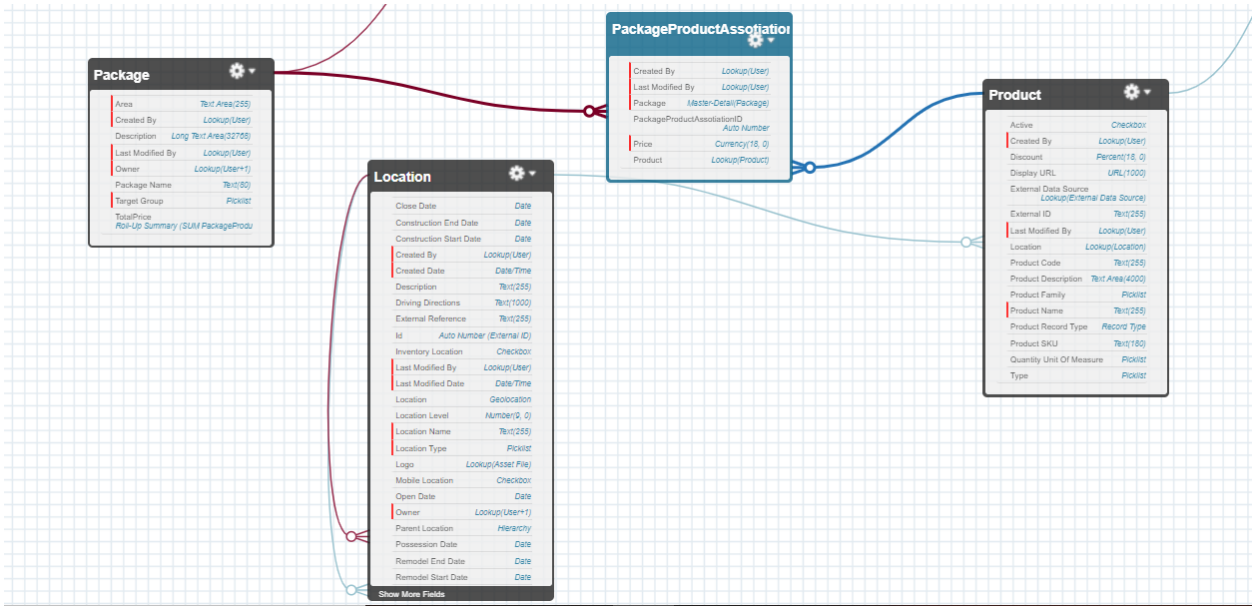


- **Product, Packages and Location Objects**

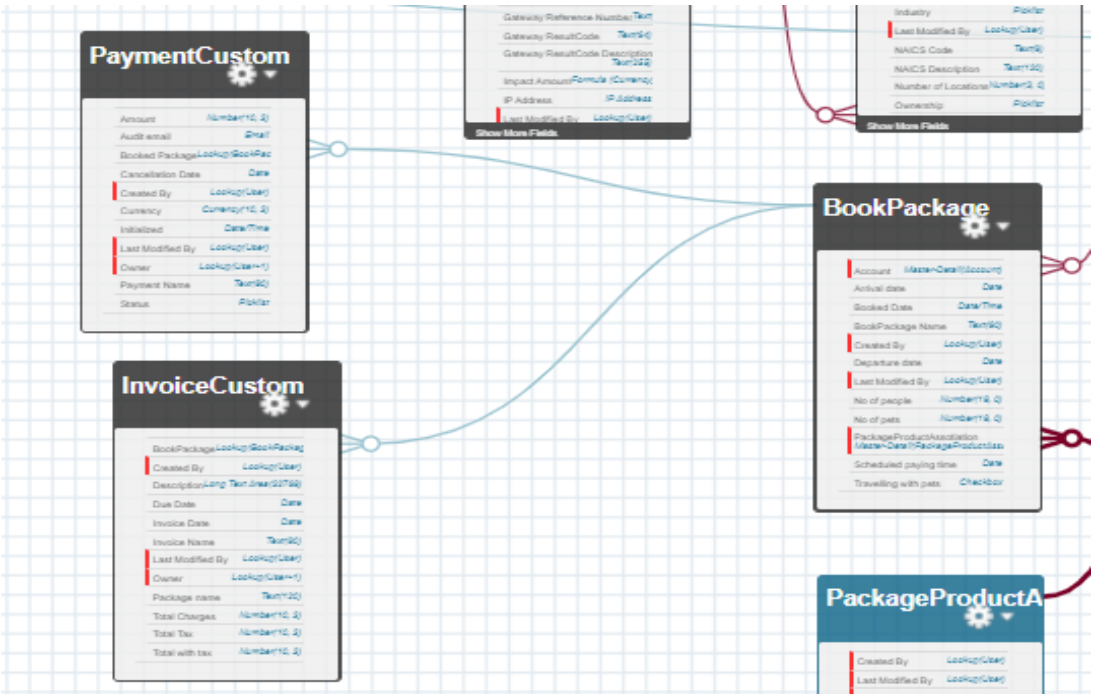
  HolidayHuraay Leisure offers their clients Products. Each product has a type: relaxing activity, accommodation, touring, meals etc, a price and a discount. Each product is linked to a single ***location***: customers can search for accommodation or outdoor activity in a specific location.

  The company offers some packages (custom objects) which contain products. Packages are targeted to a group of people: family, couples, friends. A package can contain many products e.g. a client wants accommodation in Collimar with breakfast and city tours included. A product can be included in many packages e.g. a city guided tour can be included in a family cottage escape or in a couple of caravan holidays. So between products and packages, there is a many-to-many relationship – shown in the joint custom object *PackageProductAssociation*. Since I have used the standard object Product2, I couldn't add a Master-Detail relationship with Product in *PackageProductAssociation.* I tried to replace it with a normal Lookup relationship and make it look more like a Master-Detail relationship by adding a validation rule to make the field Product2 required and a flow that automatically deletes PackageProductAssociantion related objects when a Product2 object is deleted.

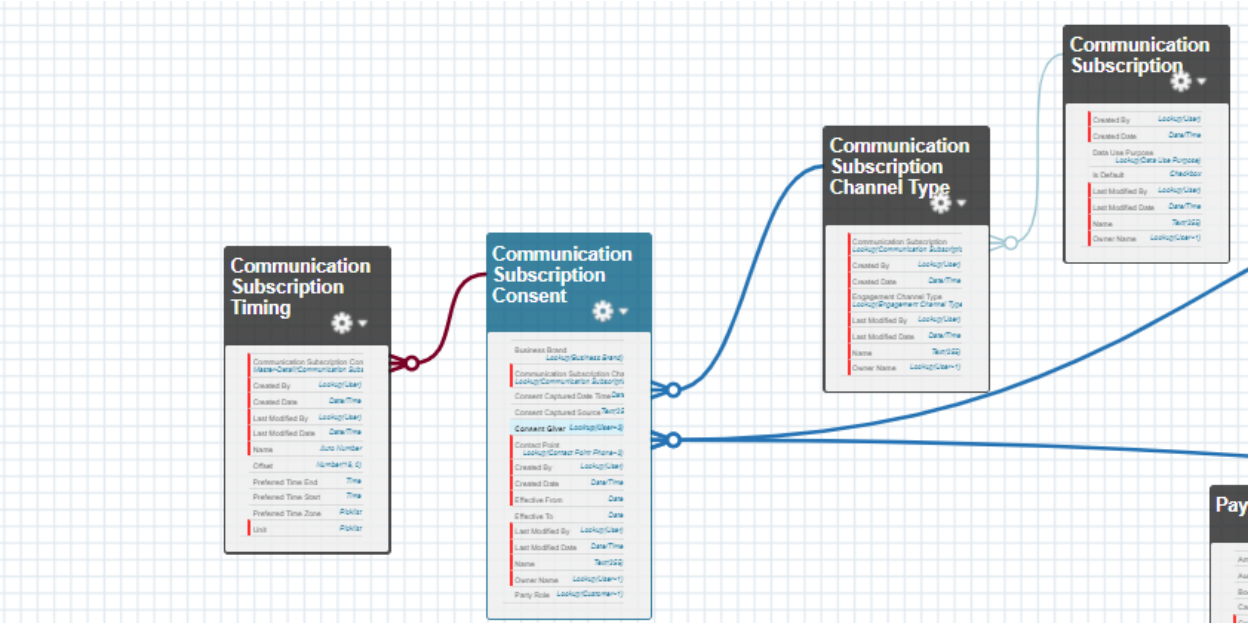
  Each package has a roll-up summary *Price* field which automatically sums all the prices from *PackageProductAssociation* (from records that have the same id as the package record).



- **Booking and Payment**
  Each customer can book a package. Between **Account** and **PackageProductAssociation** is a Many-to-Many relationship represented by the junction object **BookPackage** that contains two Master-Detail relationships. **BookPackage** object contains information about the holiday: number of pets and people, date of arrival and departure. It also contains a scheduled date for payment.
  Each Payment can be made for a BookPackage at the scheduled time. Once the payment is completed (its status change to completed) the invoice for that paid package is released.

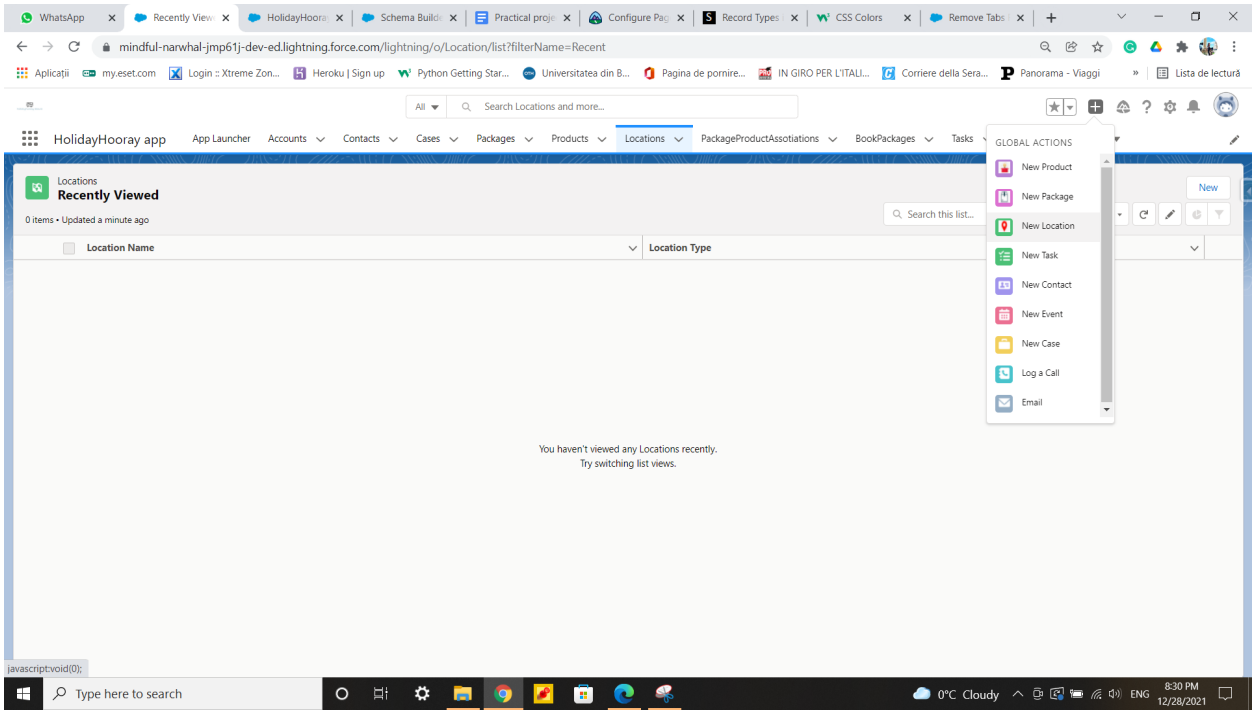


- **Communication Objects**
  - ***Communication Subscription*** to store a customer's subscription preferences for specific communication.
  - ***Communication Subscription Channel Type*** tells what communication channel – email/phone/social media- should customer service team use to contact the user. It includes users specifications from Communication Subscription.
  - ***Communication Subscription Consent*** tracks information related to each customer's consent to the communication subscription – form there the relationship Account - Communication Subscription Consent.
  - ***Communication Subscription Timing*** stores customers' favourite time intervals for being contacted by the company. For example, track whether to send a reminder to a customer a couple of days before the scheduled payment, a reminder before a trip or newsletters monthly.

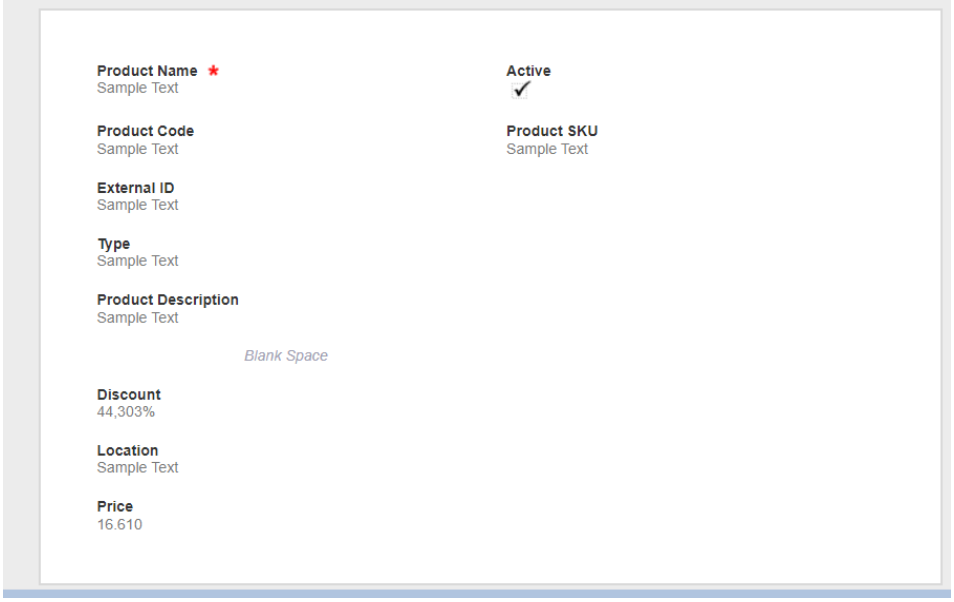


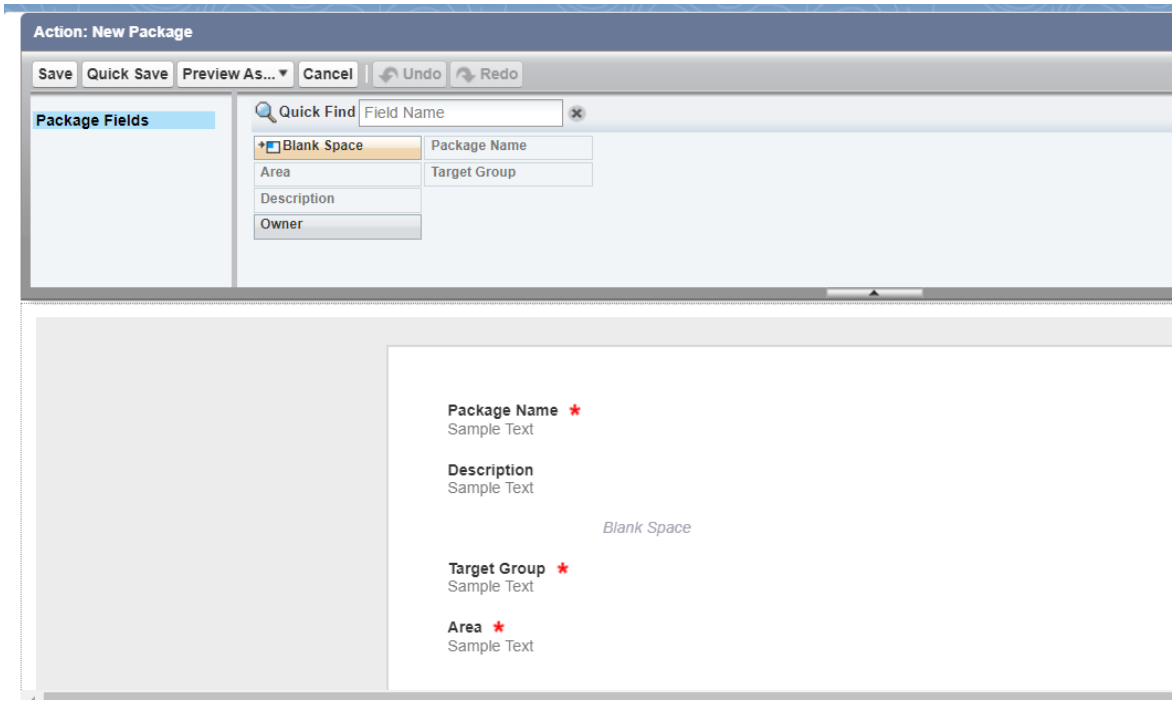
Record types and Page layouts

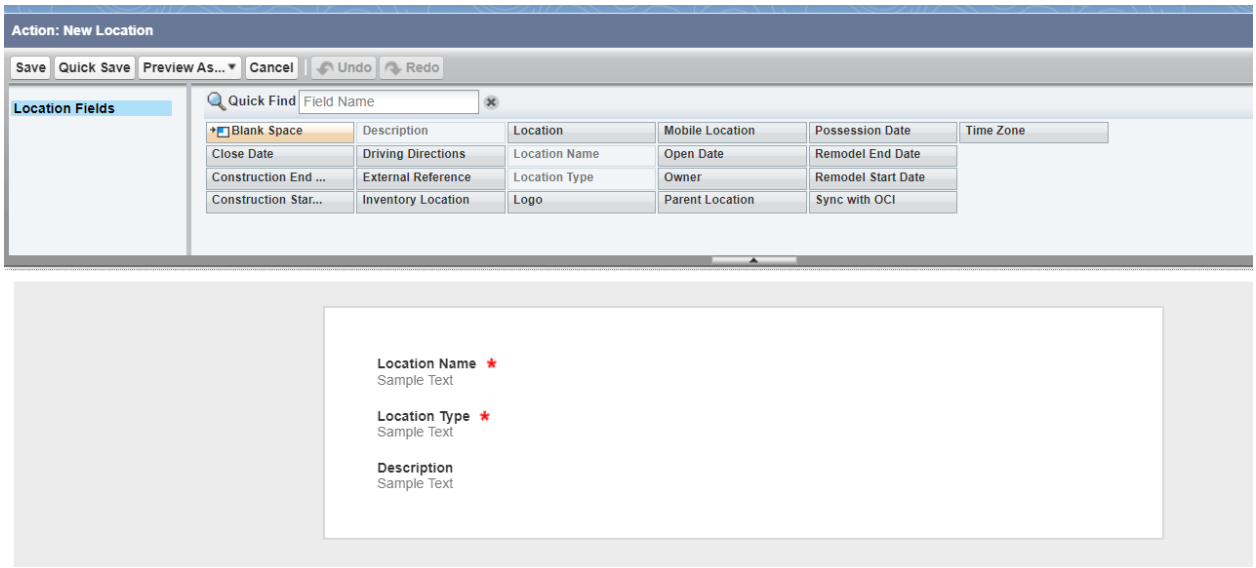
We have added some global actions.



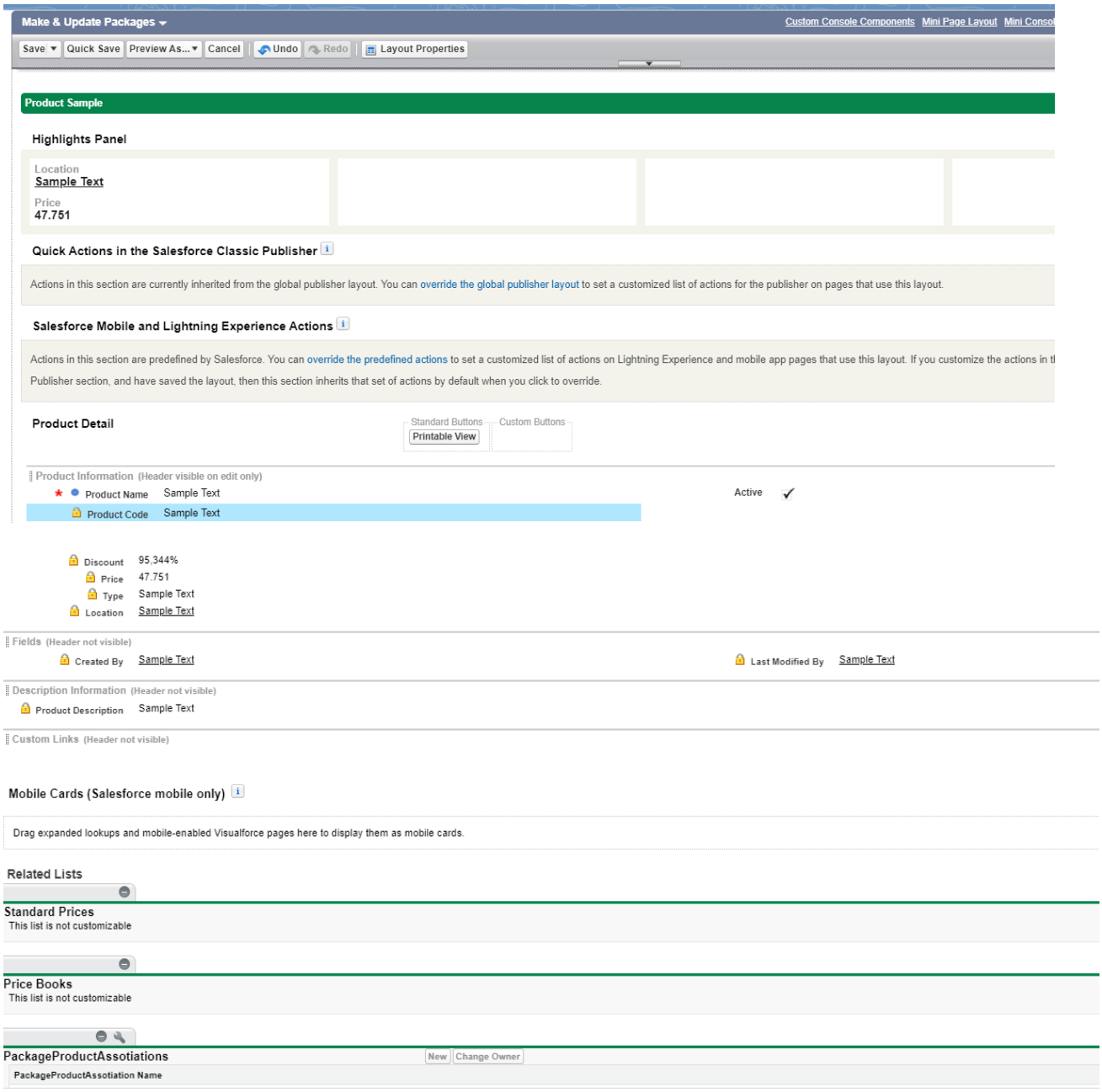
Each action has a different label:



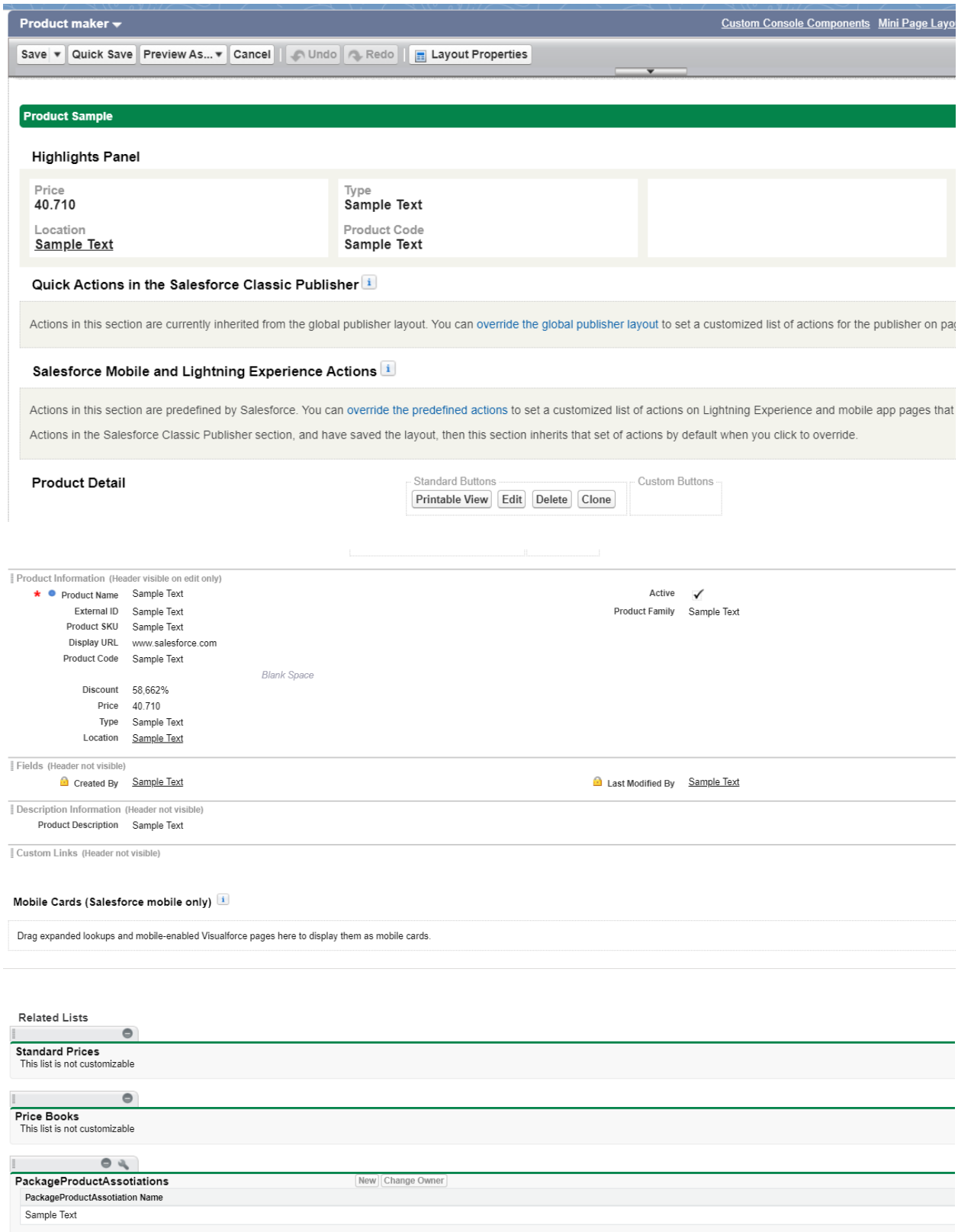

Action: New Package
Save
Quick Save
Preview As...
Cancel
Undo
Redo
Package Fields
Quick Find
Field Name
Blank Space
Package Name
Area
Target Group
Description
Owner
Package Name
Sample Text
Description
Sample Text
Blank Space
Target Group
Sample Text
Area
Sample Text


Action: New Location
Save
Quick Save
Preview As...
Cancel
Undo
Redo
Location Fields
Quick Find
Field Name
Blank Space
Description
Location
Mobile Location
Possession Date
Time Zone
Close Date
Driving Directions
Location Name
Open Date
Remodel End Date
Construction End ...
External Reference
Location Type
Owner
Remodel Start Date
Construction Star...
Inventory Location
Logo
Parent Location
Sync with OCI
Location Name
Sample Text
Location Type
Sample Text
Description
Sample Text

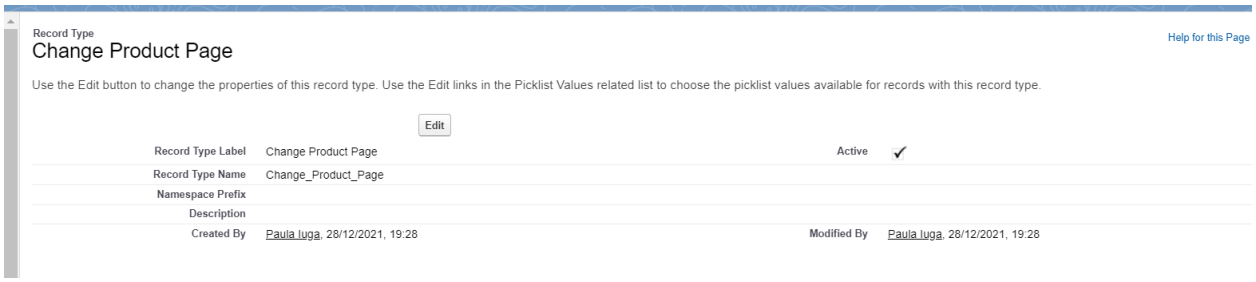
Product page layout for users with *Make & Update Packages* profile (all the fields are read-only):



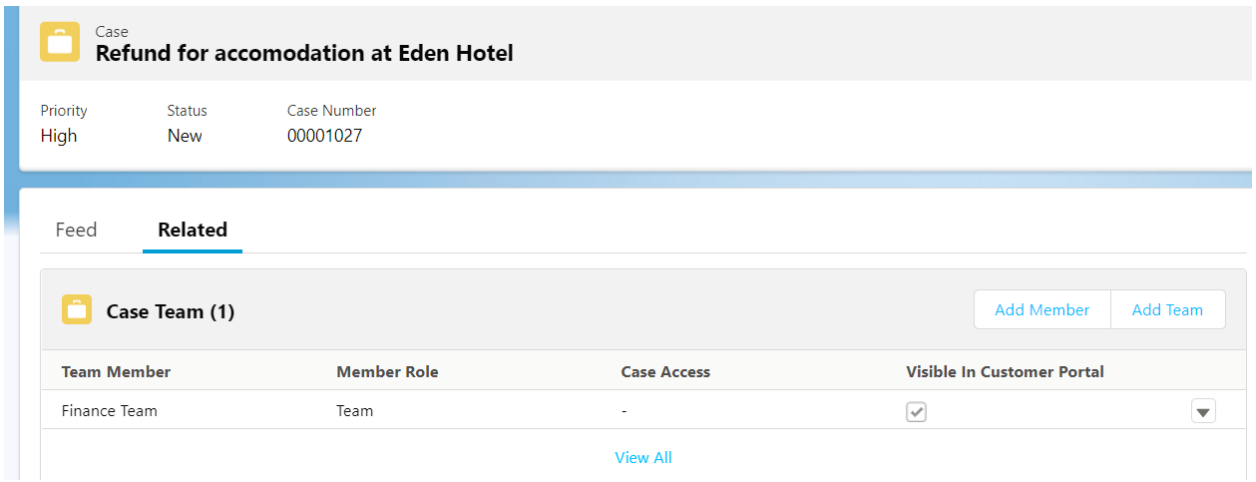
Product page layout for *Product Maker* profile:



Product record type that changes layout for *Product Maker* profile and *Make & Update Packages* profile:



Case standard page layout - we've added product and Case team.



Security Model

Profiles

- System Admin:
  - Can execute CRUD actions on any object

- Custom: Support Profile:
  - Can Read, Create, Edit, Delete cases
  - Can assign a Case Team to a case
  - Can see products related to a case

- Contract Manager:
    - Can Read, Create, Edit, Delete, View All, Modify all accounts

Custom profiles

- Make & Update Packages:
    - Can Read, Create, Edit, Delete, View All, Modify All packages
    - Can Read, Create, Edit, Delete, View All, Modify All Package Product Associations
    - Can Read, View All products/locations and booked packages so that he knows what kind of package attracts a target customers group the most
    - Can Read, View All communication objects
    - Can send offers with new packages to customers on their desired channel
    - Can Read, View All and comment on a case related to a package
    - Can see just the Service Console

- Product maker:
    - Can Read, Create, Edit, Delete, View All products and locations
    - Can Read, View All and comment on a case related to a package
    - Can see just the Service Console

- Sales Manager:
    - Can Read, Create, Edit, View All Invoices (custom ones)
    - Can Read, Create, Edit, View All Payments (custom ones)
    - Can Read, Create, Edit, View All Refunds

- Customer Manager:
    - Can Read, Create, Edit, View All Communication Objects

Role Hierarchy

- CEO Edit | Del | Assign
  - Add Role
  - Conference centre manager Edit | Del | Assign
    - Add Role
    - Event manager Edit | Del | Assign
      - Add Role
  - Software manager Edit | Del | Assign
    - Add Role
    - Web developer Edit | Del | Assign
      - Add Role
  - SVP, Customer Service & Support Manager Edit | Del | Assign
    - Add Role
    - Customer Support, International Edit | Del | Assign
      - Add Role
    - Customer Support, North America Edit | Del | Assign
      - Add Role
    - Installation & Repair Services Edit | Del | Assign
      - Add Role
  - SVP, Human Resources Edit | Del | Assign
    - Add Role
  - SVP, Sales & Marketing Edit | Del | Assign
    - Add Role
    - VP, International Sales Edit | Del | Assign
    - VP, Marketing Edit | Del | Assign
      - Add Role
      - External Relationships Edit | Del | Assign
        - Add Role
      - Social Media Agent Edit | Del | Assign
        - Add Role
    - VP, North American Sales Edit | Del | Assign
      - Add Role
      - Director, Channel Sales Edit | Del | Assign
        - Add Role
        - Channel Sales Team Edit | Del | Assign
          - Add Role
      - Director, Direct Sales Edit | Del | Assign
        - Add Role
        - Eastern Sales Team Edit | Del | Assign
          - Add Role
        - Western Sales Team Edit | Del | Assign
          - Add Role

Organization-Wide Sharing Defaults:

Object	Default Internal Access	Default External Access
Communication objects	Controlled by Parent (users perform a certain action if they can perfom it on the record associated with it)	Private (external users can't see them)
Account & Contact	Public Read/Write	Controlled by Parent
Refunds	Controlled by parent	Private
Case	Public Read/Write	Private
Package & Product	Public Read/Write	Public Read
Book Package	Public Read	Private
PackageProductAssignment	Public Read/Write	Public Read
InvoiceCustom	Public Read	Public Read
PaymentCustom	Controlled by parent	Private

Case Teams & Case Team Roles:

For better management of cases, the company wants to assign each case to a specialised team made of employees with related positions. For example, if a customer couldn't find the accommodation or their guide doesn't show up at their meeting point, their case will be assigned to the Travelling assistance team made of people with different roles such as Travel Manager, Administrative Assistant or Mobility Assistant.

Case Team Member Roles

A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T |

New

Action	Member Role Name	Case Access	Visible in Customer Portal	Modified By
Edit | Replace	Administrative Assistant	Read/Write	✓	Paula Iuga, 16/11/2021, 11:05
Edit | Replace	Finance Manager	Read/Write	✓	Paula Iuga, 16/11/2021, 11:06
Edit | Replace	Hospitality manager	Read/Write	✓	Paula Iuga, 16/11/2021, 11:07
Edit | Replace	Mobility Assistant	Read/Write	✓	Paula Iuga, 16/11/2021, 11:05
Edit | Replace	Product Expert	Read/Write	✓	Paula Iuga, 16/11/2021, 11:04
Edit | Replace	Refunds Manager	Read/Write	✓	Paula Iuga, 16/11/2021, 11:06
Edit | Replace	Sales Manager	Read/Write	✓	Paula Iuga, 16/11/2021, 11:06
Edit | Replace	Service support manager	Read/Write	✓	Paula Iuga, 16/11/2021, 11:07
Edit | Replace	Support Manager	Read/Write	✓	Paula Iuga, 16/11/2021, 10:52
Edit | Replace	Travel Manager	Read/Write	✓	Paula Iuga, 16/11/2021, 11:04

Each member is *visible in portal* so that customers can see the teams working on their cases. Each member can read and comment on a ticket.

Predefined Case Team Edit Save Cancel

Team Name: Travelling assistance team

Description:

Team Members = Required Inform

Action	Team Member		Member Role	Case Access	Visible In Customer Portal
Remove	User	Zion Parrish	Travel Manager	Read/Write	✓
Remove	User	Eve Marsh	Administrative Assistant	Read/Write	✓
Remove	User	Frida Rollins	Mobility Assistant	Read/Write	✓

Team Name: Finance Team

Description:

Team Members = Required I

Action	Team Member		Member Role	Case Access	Visible In Customer Porta
Remove	User	Alexia Whitehead	Finance Manager	Read/Write	✓
Remove	User	Karissa Fox	Refunds Manager	Read/Write	✓
Remove	User	Taylor Jenkins	Sales Manager	Read/Write	✓

Predefined Case Team Edit Save Cancel

Team Name: Service Team

Description:

Team Members = Require

Action	Team Member		Member Role	Case Access	Visible In Customer Po
Remove	User	Hanna Mckenzie	Service support manager	Read/Write	✓
Remove	User	Karissa Fox	Product Expert	Read/Write	✓

Predefined Case Team Edit Save Cancel

Team Name: Hospitality Team

Description:

Team Members = Required Informa

Action	Team Member		Member Role	Case Access	Visible In Customer Portal
Remove	User	Alexia Whitehead	Hospitality manager	Read/Write	✓
Remove	User	Jefferson Tran	Product Expert	Read/Write	✓

Data Quality

Required fields

- *Product*: Name, Price, Type (the default value is accommodation)
- *Package*: Target Group (the default value is Family), Area
- *PackageProductAssociation*: Package and product.
- *BookedPackage*: no of people, package association, arrival/departure dates, scheduled paying time, account name.
- *InvoiceCustom*: bookedPackage, package name and totals.
- *Case*: status, priority, Account Name.
- *PaymentCustom*: Amount, bookedPackage.

Validation Rules

- *Product*: Discounts can not be greater than 50% of the value of the total price.

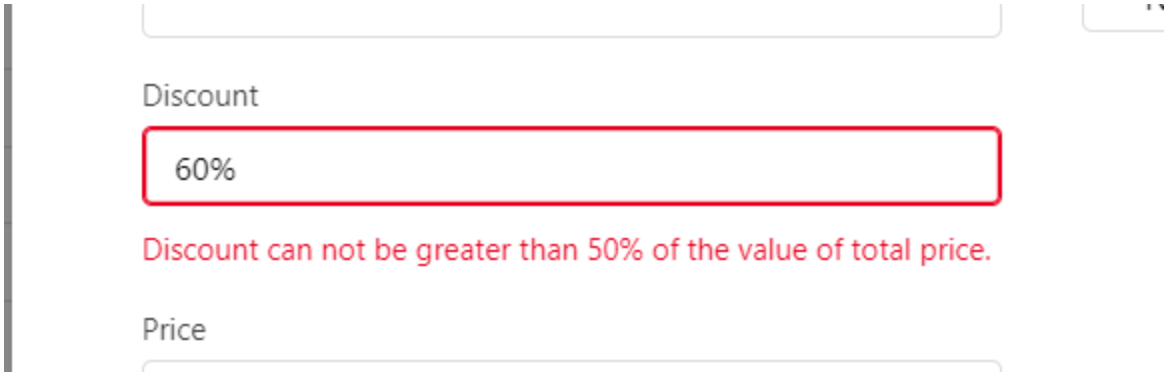
Product Validation Rule

Back to Product Validation Rules

Validation Rule Detail Edit Clone

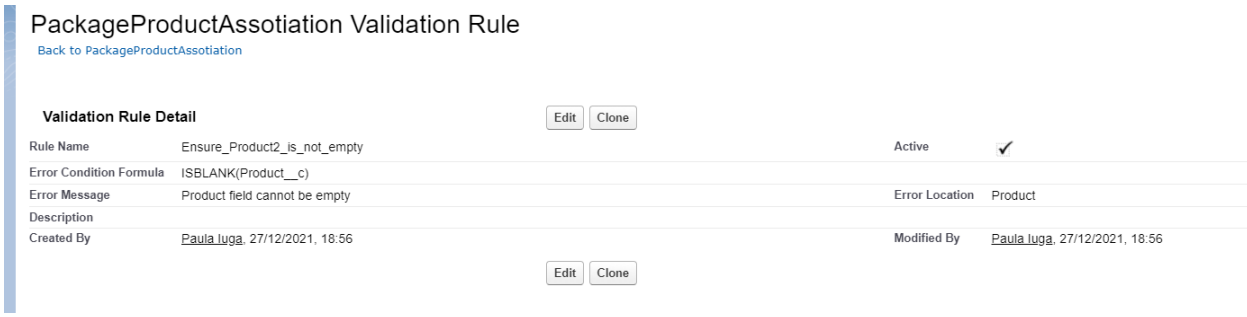
Rule Name	Discount	Active	✓
Error Condition Formula	Discount__c > 0.5		
Error Message	Discount can not be greater than 50% of the value of total price.	Error Location	Discount
Description	Discounts can not be greater than 50% of the value of the total price.		
Created By	Paula Iuga, 15/11/2021, 14:55	Modified By	Paula Iuga, 29/12/2021, 10:27

Edit Clone



- *PackageProductAssotiation:*

Since I couldn't make the Product required on the lookup relationship (I received the following error: *Cannot add a lookup relationship child with cascade or restrict options to Product2 (Related field: Domain)*), I used a validation rule to ensure that Product Field is not empty when creating/editing a PackageProductAssotiation object.



PackageProductAssotiation Validation Rule

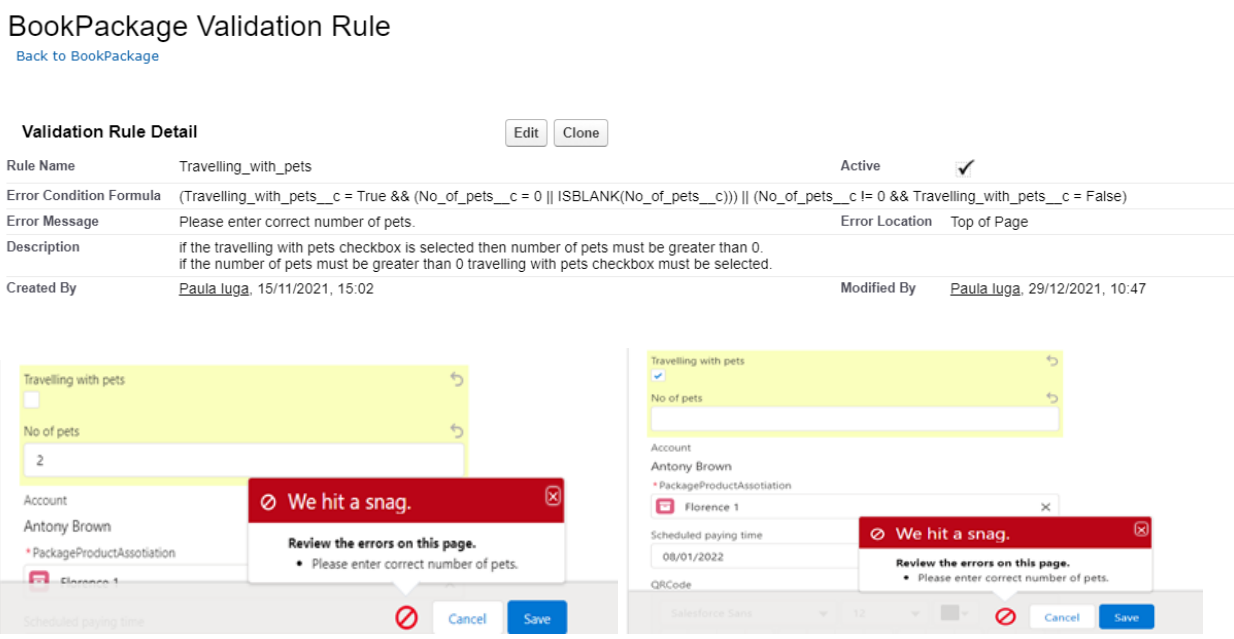
Back to PackageProductAssotiation

Validation Rule Detail Edit Clone

Rule Name	Ensure_Product2_is_not_empty	Active	✓
Error Condition Formula	ISBLANK(Product__c)		
Error Message	Product field cannot be empty	Error Location	Product
Description			
Created By	Paula Iuga, 27/12/2021, 18:56	Modified By	Paula Iuga, 27/12/2021, 18:56

Edit Clone

- *BookPackage*:
  - If the *travelling with pets* checkbox is selected then the *number of pets* must be greater than 0 (default value is 0).
  - If the *number of pets* must be greater than 0 *travelling with pets* checkbox must be selected.



BookPackage Validation Rule

Back to BookPackage

Validation Rule Detail Edit Clone

Rule Name	Travelling_with_pets	Active	✓
Error Condition Formula	(Travelling_with_pets__c = True && (No_of_pets__c = 0 || ISBLANK(No_of_pets__c))) || (No_of_pets__c != 0 && Travelling_with_pets__c = False)		
Error Message	Please enter correct number of pets.	Error Location	Top of Page
Description	if the travelling with pets checkbox is selected then number of pets must be greater than 0. if the number of pets must be greater than 0 travelling with pets checkbox must be selected.		
Created By	Paula Iuga, 15/11/2021, 15:02	Modified By	Paula Iuga, 29/12/2021, 10:47

- *InvoiceCustom*: Invoice date must be greater than the scheduled payment time.

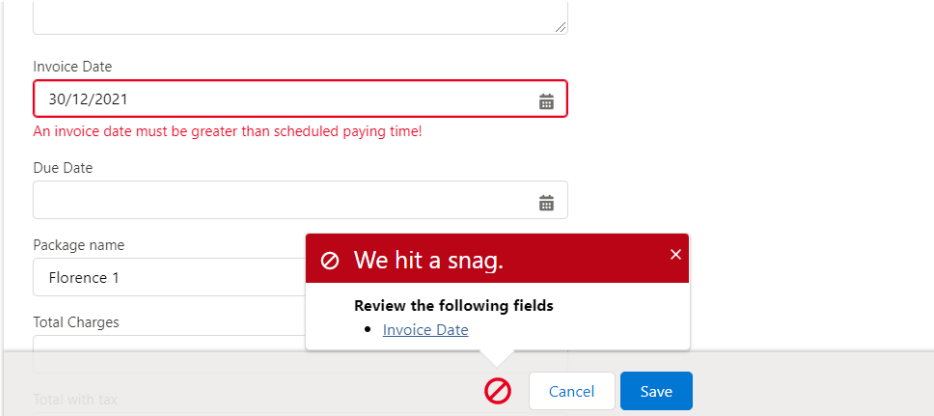
InvoiceCustom Validation Rule

Back to InvoiceCustom

Validation Rule Detail Edit Clone

Rule Name	Invoice_date	Active	✓
Error Condition Formula	Invoice_Date__c < BookPackage__r.Scheduled_paying_time__c		
Error Message	An invoice date must be greater than scheduled paying time!	Error Location	Invoice Date
Description	An invoice date must be greater than scheduled paying time.		
Created By	Paula Iuga, 15/11/2021, 14:47	Modified By	Paula Iuga, 29/12/2021, 10:58

Edit Clone



- *PaymentCustom*: Any payment with a cancellation date greater than the current date must-have status: cancelled.

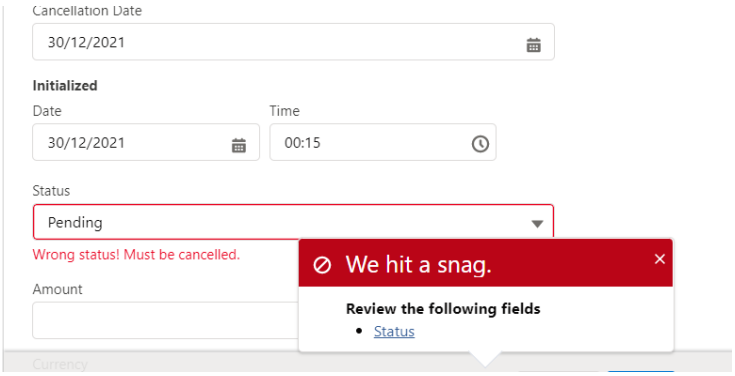
PaymentCustom Validation Rule

Back to PaymentCustom

Validation Rule Detail Edit Clone

Rule Name	Payment_date	Active	✓
Error Condition Formula	(Cancellation_Date__c > TODAY()) && !ISPICKVAL(Status__c, "Cancelled")		
Error Message	Wrong status! Must be cancelled.	Error Location	Status
Description	Any payment with cancelation date greater than current date must have status cancelled.		
Created By	Paula Iuga, 15/11/2021, 14:53	Modified By	Paula Iuga, 29/12/2021, 11:04

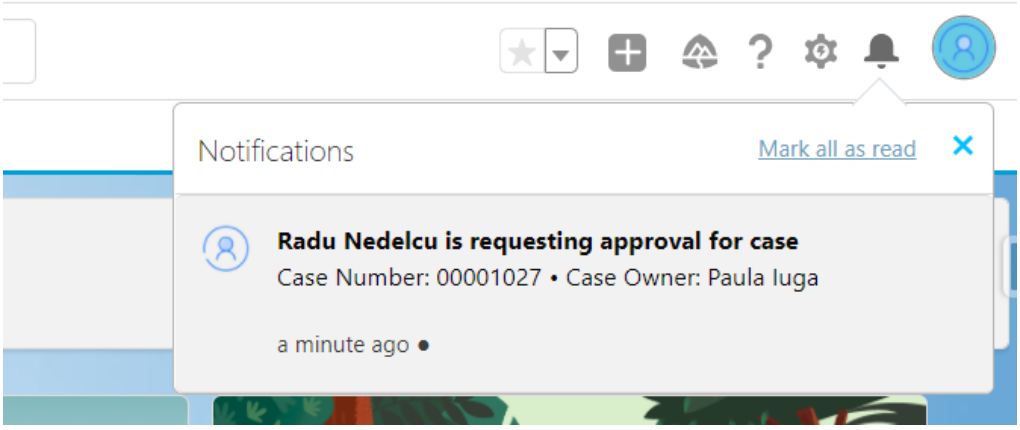
Edit Clone

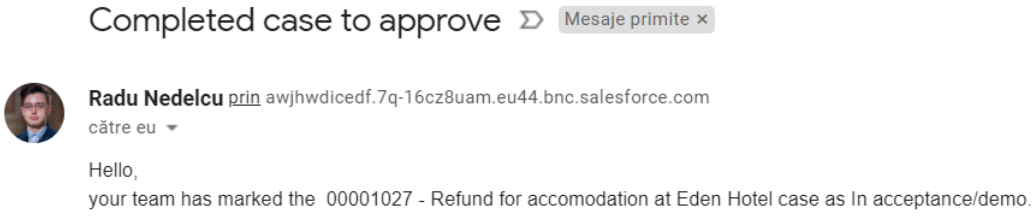


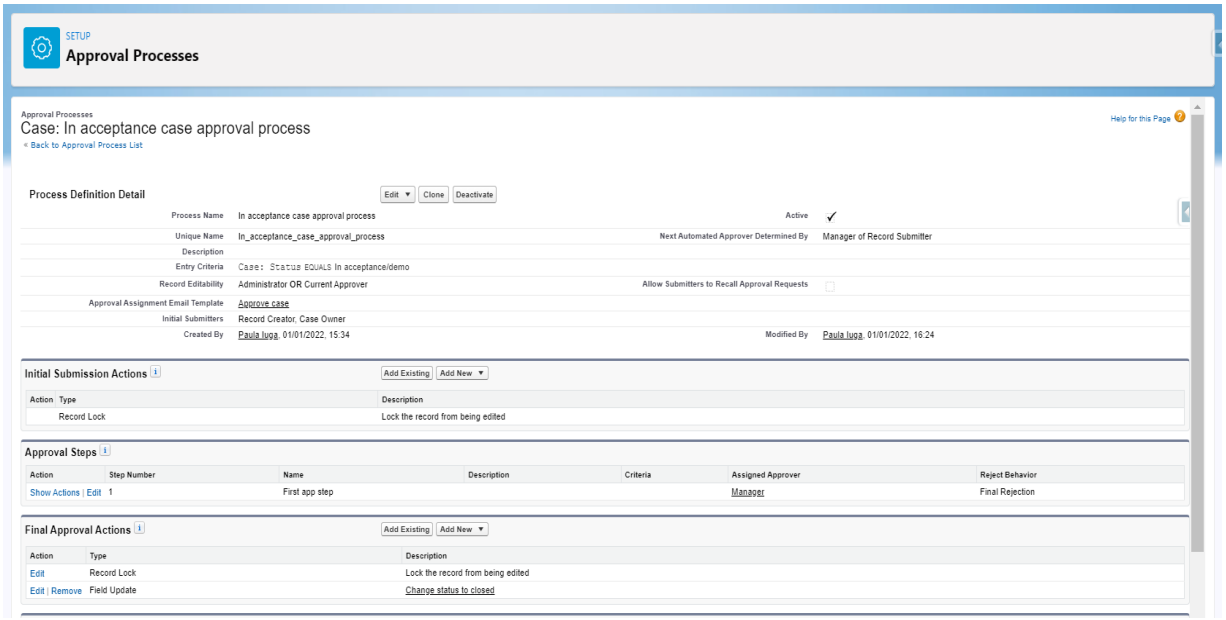
Process Automation

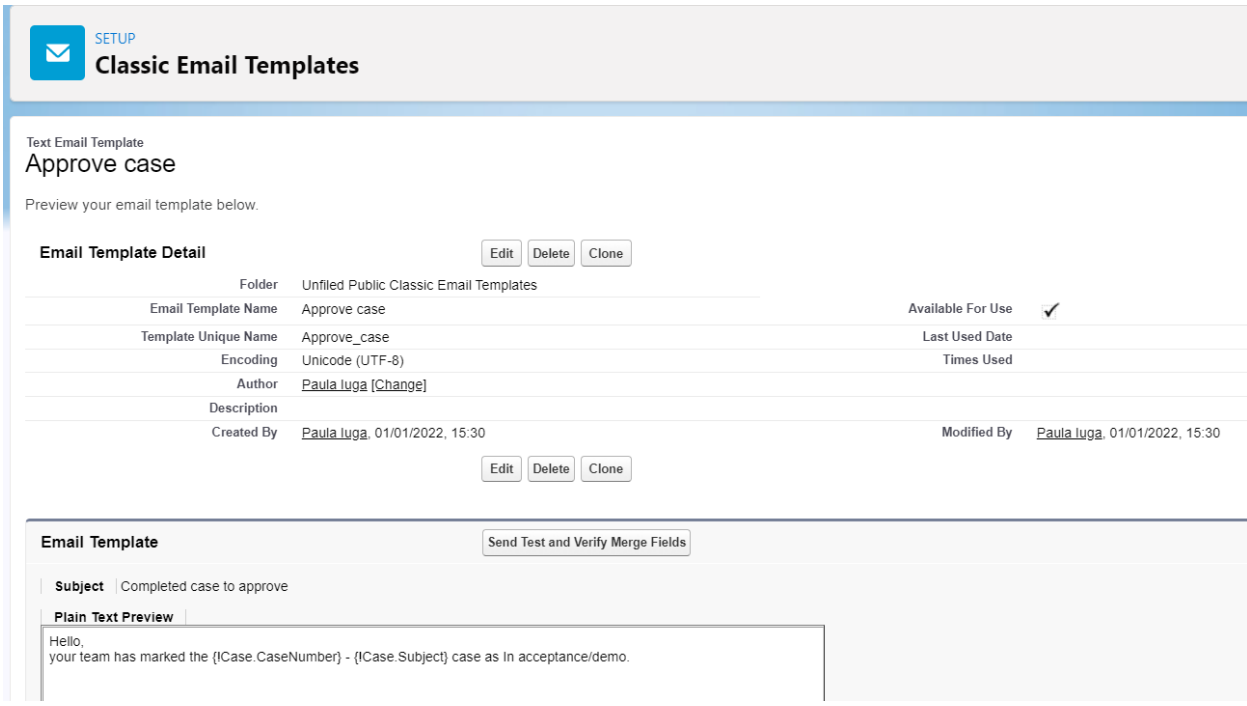
- **Approval Process for cases:**

When a user updates the status of a case to *In acceptance/demo*, the record should be sent to approval to the owner's manager. He/she should receive an email alert and a notification. When he/she approves the case, its' status changes automatically to *Closed*.







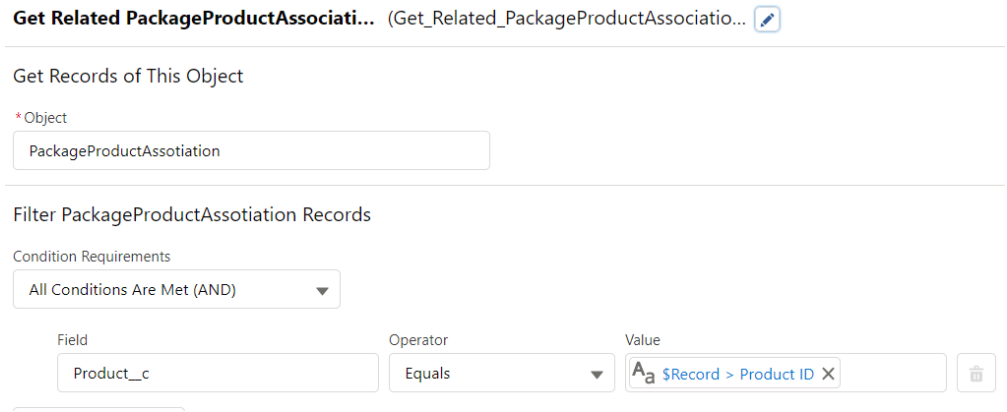


- **Delete PackageProductAssociation records when related Product records are deleted flow:**

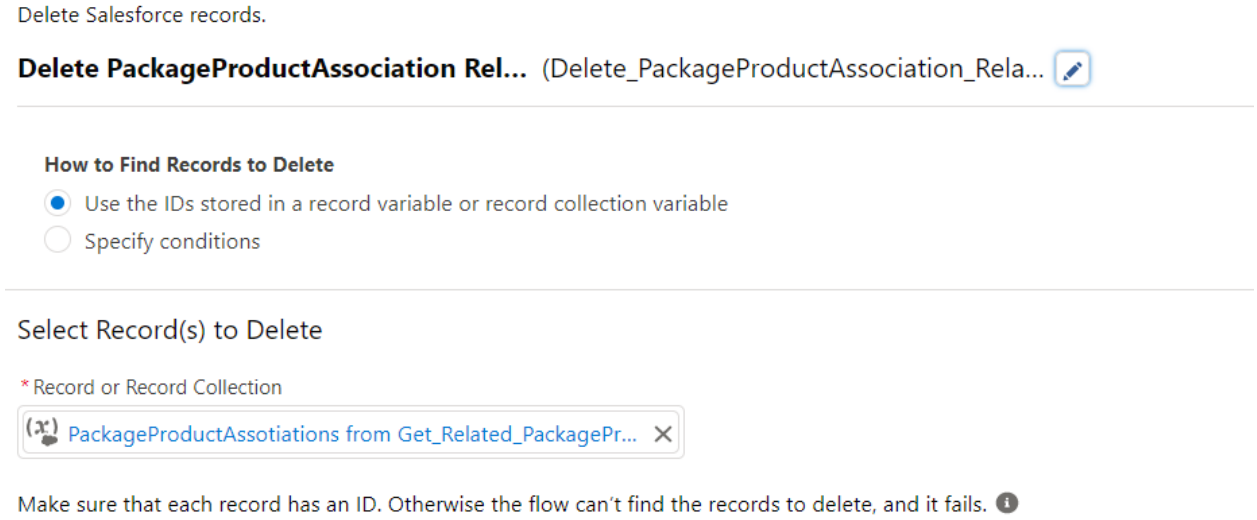
  To create the flow:

  For this should be created a *Record-Triggered Flow*.
  Selected Object is *Product2*.
  Trigger the Flow When: *A record is deleted*.
  Condition Requirements: *None*.

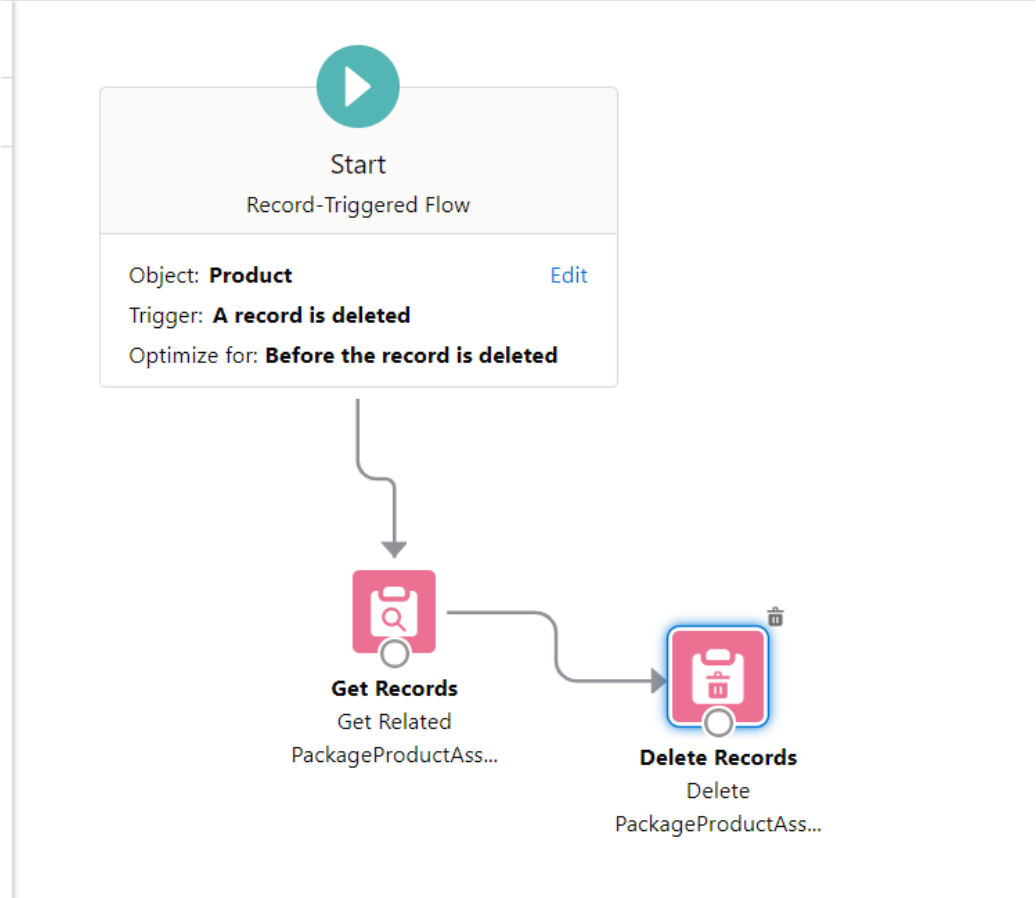
When the flow is triggered it should first get all related PackageProductAssociation records:



Then those records should be deleted:



In the end, the flow should look like this:

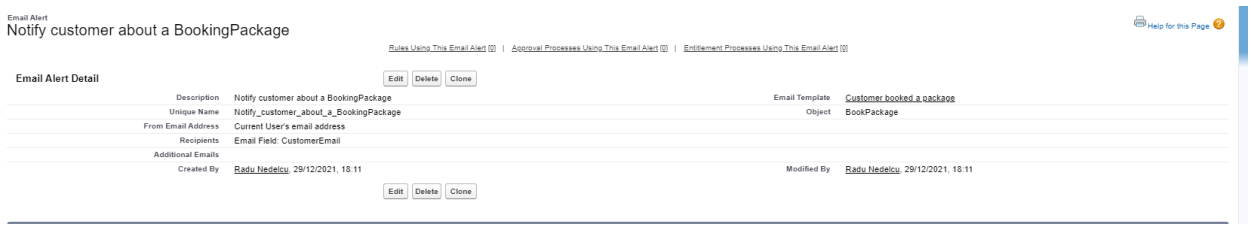


- **Booking a Package Process Builder:**
  When booking a package, the user should be notified that he booked a package and should be notified again 2 days before the scheduled payment.

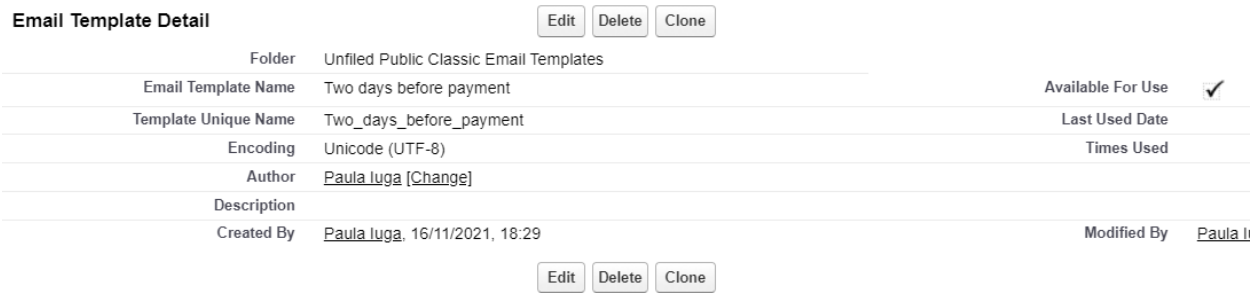
  The process starts when – *a record changes*
  Object to start the process is *BookPackage* and the process starts *only when a record is created.*

  Action email alert: for this, I have added a new email template

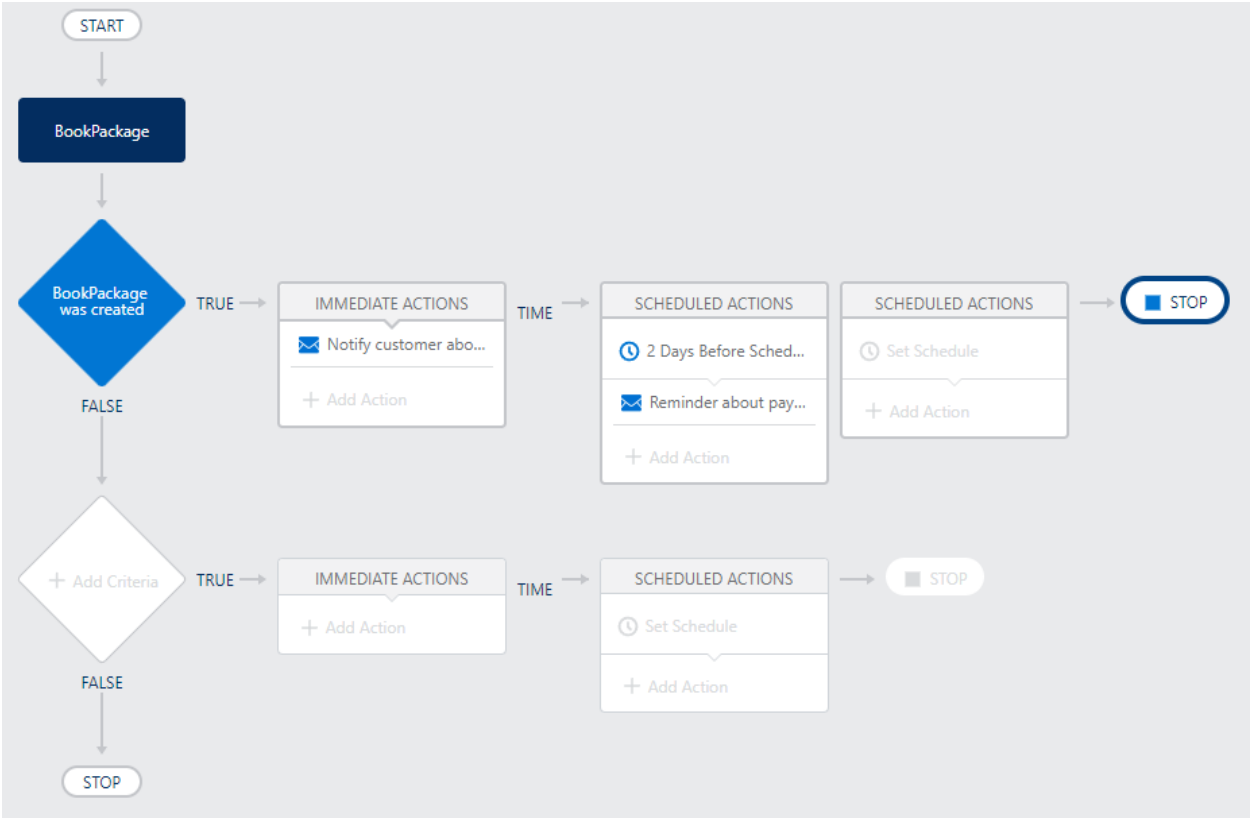
  And a new email alert:

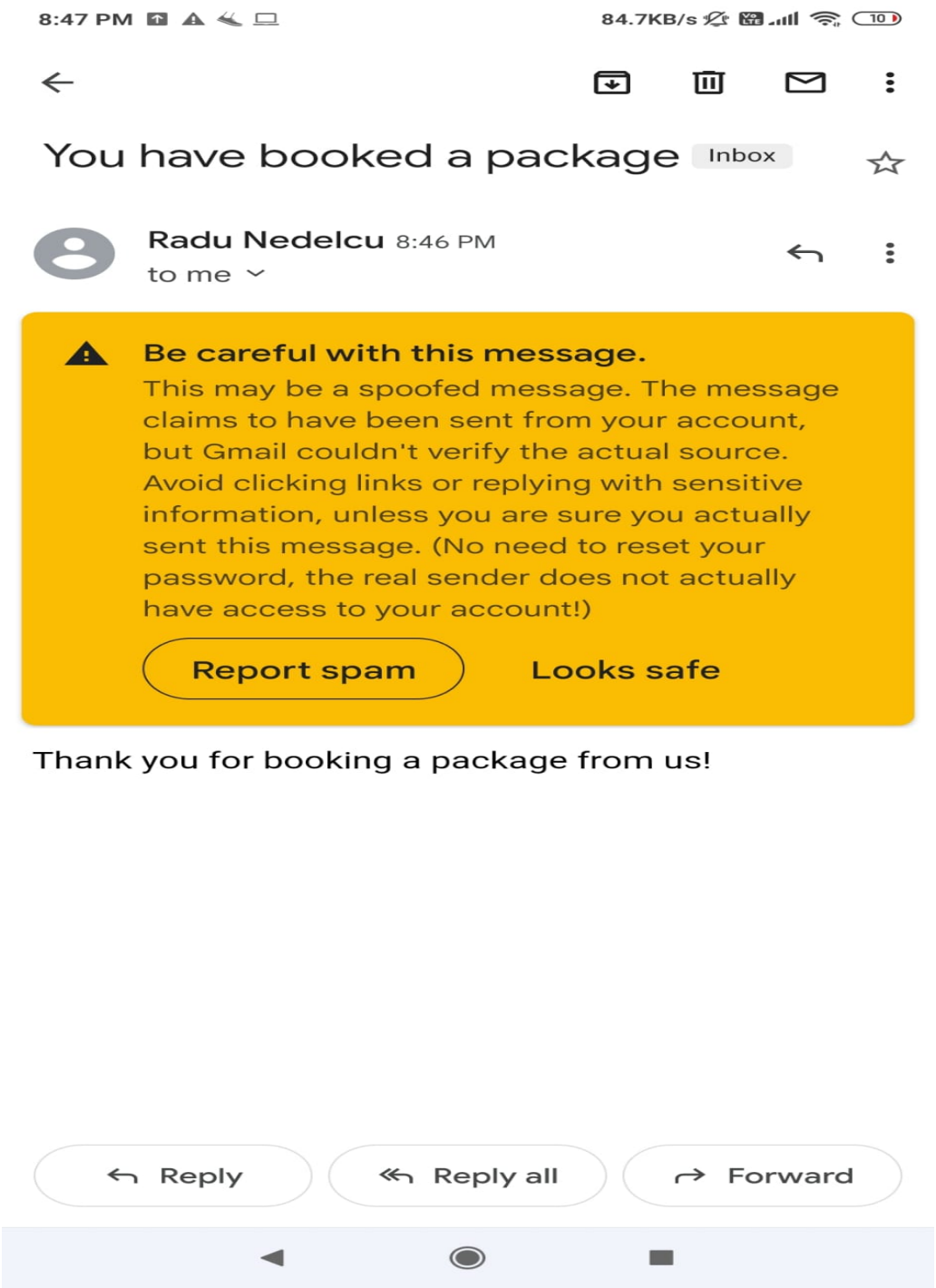


For the 2 days before the scheduled payment notification, I have scheduled another email alert.



In the end the process should look like this:




8:47 PM
84.7KB/s
You have booked a package
Inbox
Radu Nedelcu 8:46 PM
to me
Be careful with this message.
This may be a spoofed message. The message claims to have been sent from your account, but Gmail couldn't verify the actual source. Avoid clicking links or replying with sensitive information, unless you are sure you actually sent this message. (No need to reset your password, the real sender does not actually have access to your account!)
Report spam
Looks safe
Thank you for booking a package from us!
Reply
Reply all
Forward

Custom code automations

Generate Invoice

When a payment updates its status to Completed an invoice should be automatically generated. Its total charges should be set to the total amount of the package bought. Each package has a 10% tax rate. After calculating the Total Tax, the Total with Tax field should be updated too.

The Trigger should be on the PaymentCustom Object. It should be an after update trigger because we want to create a new Invoice and link it to the BookPackage after the payment has been successfully made.

```
trigger HH_Payment_Trigger on Payment__c (before update, after update) {

    HH_Payment_Trigger_Handler handler = new HH_Payment_Trigger_Handler();

    if(Trigger.isBefore && Trigger.isUpdate)
        handler.handlePaymentBeforeUpdate(Trigger.newMap, Trigger.old);

    if(Trigger.isAfter && Trigger.isUpdate)
        handler.handlePaymentAfterUpdate(Trigger.new);


}
```

```
public class HH_Payment_Trigger_Handler {
    private final string BEFORE_UPDATE_ERROR = 'Completed payments cannot be edited';

    public void handlePaymentAfterUpdate(List<Payment__c> newPayments) {
        List<Invoice__c> invoicesToInsert = new List<Invoice__c>();
        String packageName, customerName;
        Double totalSum;
        Payment__c tmp;
        Map<ID,Payment__c> payments = new Map<ID,Payment__c>([SELECT Booked_Package__r.Name, Booked_Package__r.Account__r.Name, Booked_Package__r.PackageBooked__r.TotalPrice__c  FROM Pa
        for(Payment__c p: newPayments) {
            if(p.Status__c == 'Completed') {
              tmp = payments.get(p.Id);
              packageName = tmp.Booked_Package__r.Name;
              customerName = tmp.Booked_Package__r.Account__r.Name;
              totalSum = tmp.Booked_Package__r.PackageBooked__r.TotalPrice__c;

              invoicesToInsert.add( generateInvoice(packageName, customerName, totalSum) );
            }
         }

        insert invoicesToInsert;
    }

    private Invoice__c generateInvoice(String packageName, String customerName, Double totalSum) {
        Invoice__c invoice = new Invoice__c();
        invoice.Invoice_Date__c = System.today();
        invoice.Package_name__c = packageName;
        invoice.Name = customerName + ' - ' + packageName.toUppercase();
        invoice.TotalWithTax__c = totalSum;
        invoice.Total_Tax__c = 0.1;
        invoice.TotalCharges__c = totalSum - totalSum * 0.1;

        return invoice;
    }
```

- **Completed payments should not be editable**

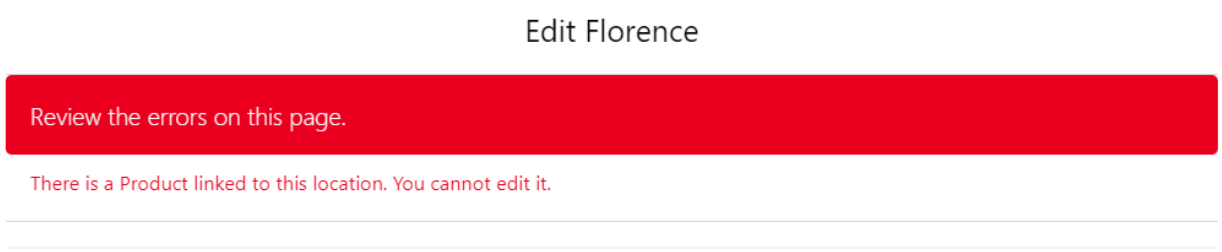
  The Trigger should be on the PaymentCustom Object. It should be an before update trigger because we want to cancel the update display an error if the payment's status is 'Completed'

```
public void handlePaymentBeforeUpdate(Map<ID, Payment__c> newPayments, List<Payment__c> oldPayments) {
for(Payment__c pay : oldPayments)
    if(pay.Status__c == 'Completed')
        newPayments.get(pay.ID).addError(BEFORE_UPDATE_ERROR);
}
```

- **Location Error**

  If a location record related to one or more products is edited then the user should receive an error.

  The Trigger should be on Location Object. It should be a before update trigger because we want to block the user to update a location that is linked to a product.



HH_Location_Trigger_Handler.apxc | HH_Location_Trigger.apxt

Code Coverage: None | API Version: 53

```
public class HH_Location_Trigger_Handler {
    public void handleLocationBeforeUpdate(List<Schema.Location> locations) {

        List<Product2> usedLocations = [SELECT Location__r.Id__c from Product2];
        List<String> usedIds = new List<String>();

        for(Product2 p : usedLocations) {
            usedIds.add(p.Location__r.Id__c);
        }

        for(Schema.Location l : locations) {
            if(usedIds.contains(l.Id__c)) {
                l.addError('There is a Product linked to this location. You cannot edit it.');
            }
        }


    }

}
```

HH_Location_Trigger_Handler.apxc | HH_Location_Trigger.apxt

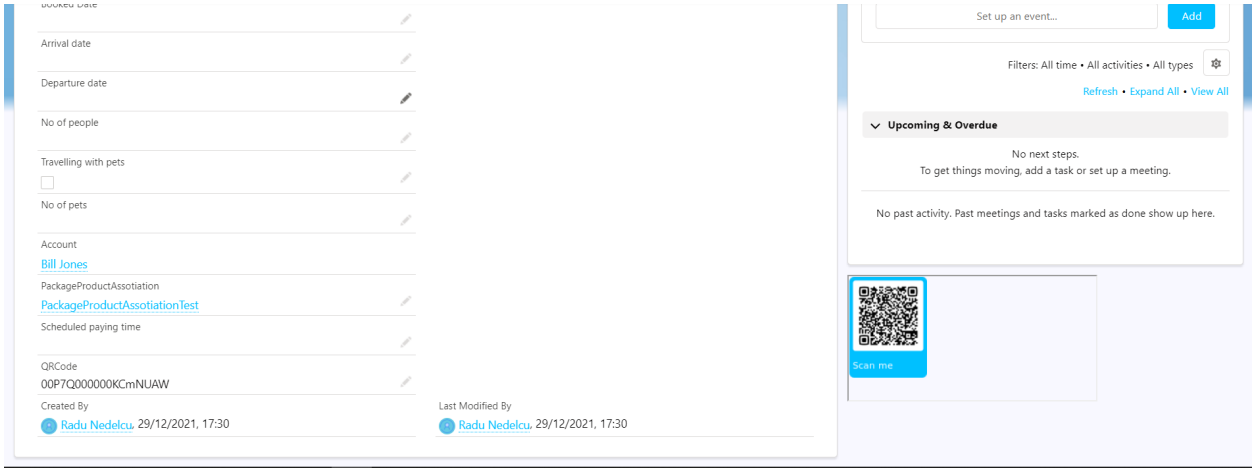
Code Coverage: None | API Version: 53

```
trigger HH_Location_Trigger on Schema.Location (before update) {

   HH_Location_Trigger_Handler trigger_handler =  new HH_Location_Trigger_Handler();

    if(Trigger.isBefore && Trigger.isUpdate) {
       trigger_handler.handleLocationBeforeUpdate(Trigger.new);
    }

}
```

Integration

HolidayHuraay Leisure would like to improve their customers' journey by giving them electronic tickets - this way the receptionist can just scan their QR code and get all the necessary details in order to complete their order. This could be done with REST Callouts (Salesforce gets data from another web service). I would choose REST, not SOAP Callouts because REST are faster and easier. To use SOAP Callouts a developer needs to generate an Apex Class from the WSDL - this can be done automatically but because the WDSL parser is difficult developers could come to write that class by hand.

It's easier to use REST Callouts, a developer just has to set the EndPoint and the Method – POST in this case because we want to send data to the QR Generator API then receive the code which is then going to be saved as an attachment. In order to show the right QR code from the database, we can just store the attachment ID in a field on the BookingRecord object and use LWC to display it in the view. I would use Named Credential to set an endpoint this way I don't usually need to have credentials in code and whitelist the endpoint. Moreover, credentials stored using Name Credentials are encrypted by auto-creating org-specific keys. Unfortunately, the API used needs to receive the token in the URL, not in the header/body of the request, so it is necessary to store the token in code too.




```
trigger BookPackageTrigger on BookPackage__c (after insert) {
    if(Trigger.isAfter && Trigger.isInsert)
        BookPackageTriggerHandler.handleBookPackageAfterInsert(Trigger.new);
}
```

```
public class BookPackageTriggerHandler {
    private static final string API_TOKEN = 'YQpUgTQW35Ikvj5qkdsgJ2g2tGyo0vjTcxP2mPpAUUL46aPiw4-YARKfjQ-L2rJJ';
    public static void handleBookPackageAfterInsert(List<BookPackage__c> packages) {
        List<ID> bpIDs = new List<ID>();
        for(BookPackage__c bp : packages)
            bpIDs.add(bp.Id);

        doQRCallout(bpIDs);
    }

    @future(callout=true)
    private static void doQRCallout(List<ID> bpIds) {
        Map<Id,BookPackage__c> bps = new Map<Id,BookPackage__c>([SELECT ID FROM BookPackage__c WHERE ID in :bpIds]); //Get the BookPackages that need QR code to be generated
        List<Attachment> attachments = new List<Attachment>(); //Bulkify the insertion of QR codes
        //Prep the request
        HttpRequest request = new HttpRequest();
        request.setMethod('POST');
        request.setTimeout(3000);
        request.setHeader('Content-Type', 'application/json');
        request.setEndpoint('callout:QR_Generator_API?access-token='+ API_TOKEN);
        QRParams params = new QRParams();


        for(Id bpId : bps.keySet()) { //Generate QR code for each BookPackage
         params.qr_code_text = 'https://mindful-narwhal-jmp61j-dev-ed.lightning.force.com/lightning/r/BookPackage__c/'+ bpId + '/view';
         request.setBody(JSON.serialize(params));
         HttpResponse response = new Http().send(request);
         //Get the QR as a blob
         Blob blobbedResp = response.getBodyAsBlob();
         Attachment attach = new Attachment();
         attach.Body = blobbedResp;
         attach.ParentId = bpId;
         attach.ContentType = 'image/png';
         attach.Name = 'QR Code of '+bpId+'.png';
         attachments.add(attach);
        }

        insert attachments;
        for(Attachment attach : attachments)
            bps.get(attach.ParentID).QRCode__c = attach.Id;

        update bps.values();
}
    private class QRParams{
        private String frame_name = 'bottom-frame';
        private String qr_code_text;
        private String image_format = 'PNG';
        private String frame_color = '#02bfff';
        private String frame_text_color = '#ffffff';
        private String frame_text = 'Scan me';
        private String marker_left_template =  'version13';
        private String marker_right_template = 'version13';
        private String marker_bottom_template = 'version13';
    }
}
```

Lightning Web Components

- ### QR Code visualization

  Using templates we can control what the user sees: if the QR code was not generated yet, we display a message to come back later. If the QR was generated, then we also have an URL available for it which we obtain by combining the current record's QR_FIELD and the URL for the image servlet.

qRCode.html | qRCode.js | qRCode.js-meta.xml

force-app > main > default > lwc > qRCode > qRCode.html > template

```
<template>
    <template if:true={url}>
        <iframe src={url}></iframe>
    </template>
    <template if:false={url}>
        <lightning-card>
            The QR Code was not generated yet. Please try again in a few minutes.
        </lightning-card>
    </template>
</template>
```

qRCode.html | qRCode.js | qRCode.js-meta.xml

force-app > main > default > lwc > qRCode > qRCode.js > QRCode > (get) url

```
import { LightningElement, wire, api } from 'lwc';
import { getRecord, getFieldValue } from "lightning/uiRecordApi";
import QR_FIELD from "@salesforce/schema/BookPackage__c.QRCode__c";
export default class QRCode extends LightningElement {
    @api recordId;

    @wire(getRecord, {
        recordId: "$recordId",
        fields: [QR_FIELD]
      })
      record;

    get url() {
        return getFieldValue(this.record.data, QR_FIELD) ?
        "https://mindful-narwhal-jmp61j-dev-ed--c.documentforce.com/servlet/servlet.FileDownload?file=" + getFieldValue(this.record.data, QR_FIELD) : '';
    }


}
```

qRCode.html | qRCode.js | qRCode.js-meta.xml

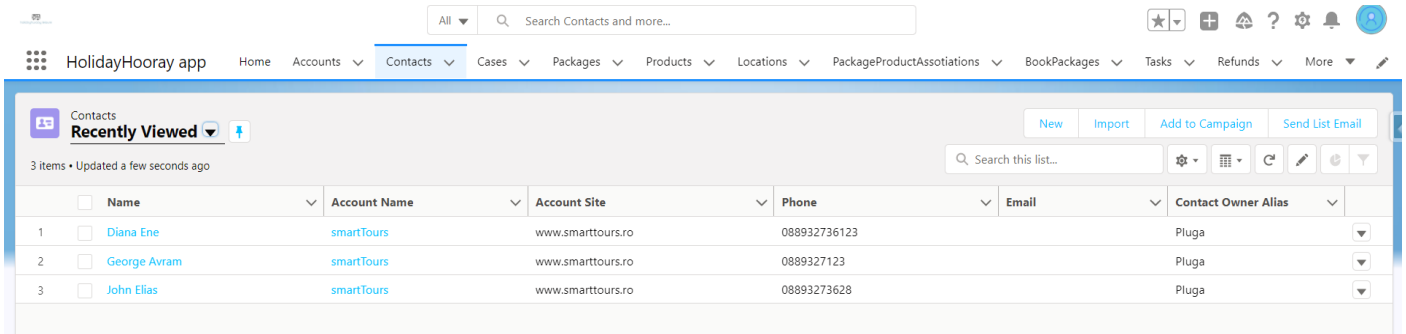
force-app > main > default > lwc > qRCode > qRCode.js-meta.xml > LightningComponentBundle > targets

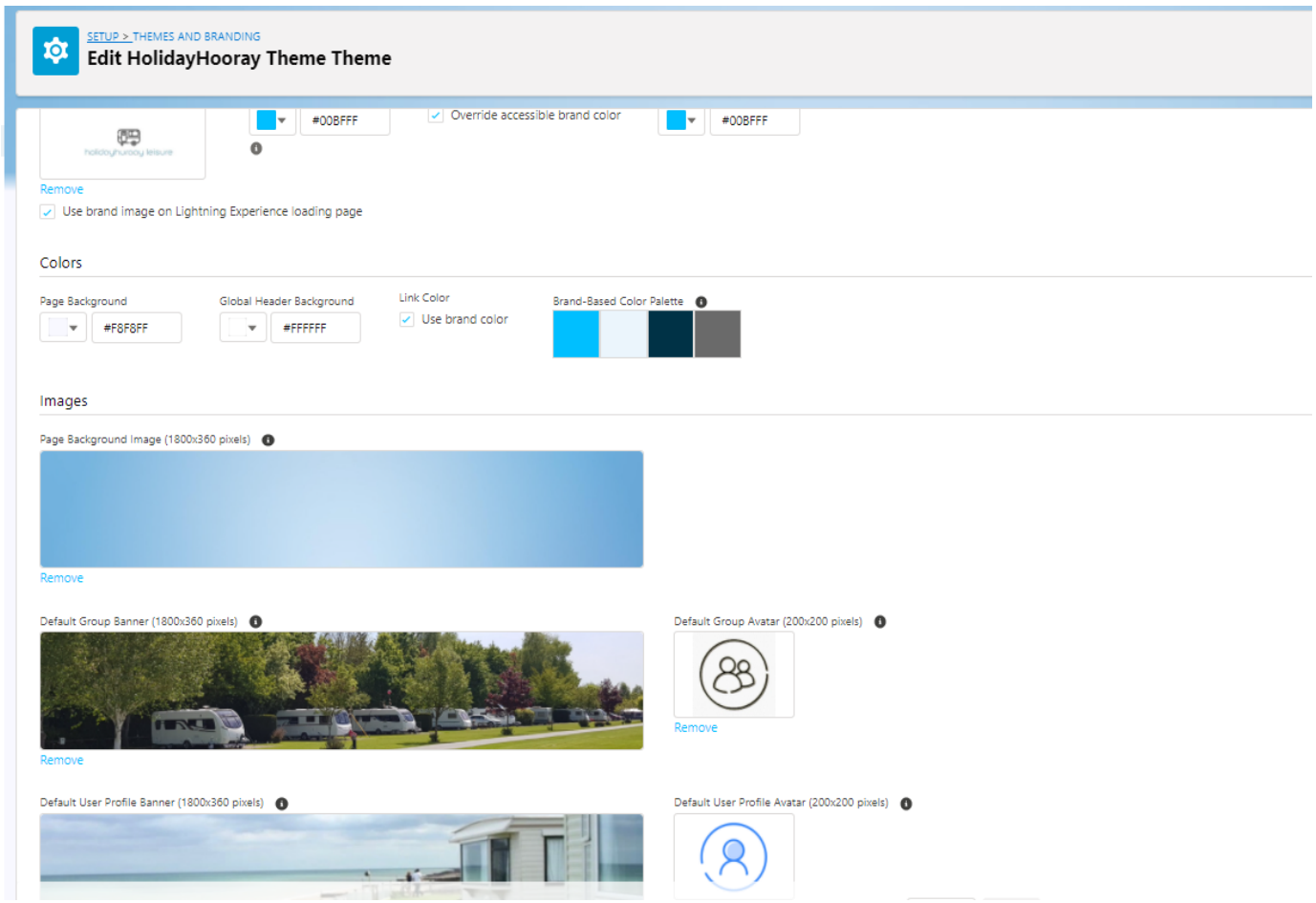
```
<?xml version="1.0" encoding="UTF-8"?>
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
    <apiVersion>52.0</apiVersion>
    <isExposed>true</isExposed>
    <targets>

        <target>lightning__AppPage</target>
        <target>lightning__RecordPage</target>
        <target>lightning__HomePage</target>
    </targets>
</LightningComponentBundle>
```

Branded UI

We've created a new Lightning Theme - HolidayHorray Theme.

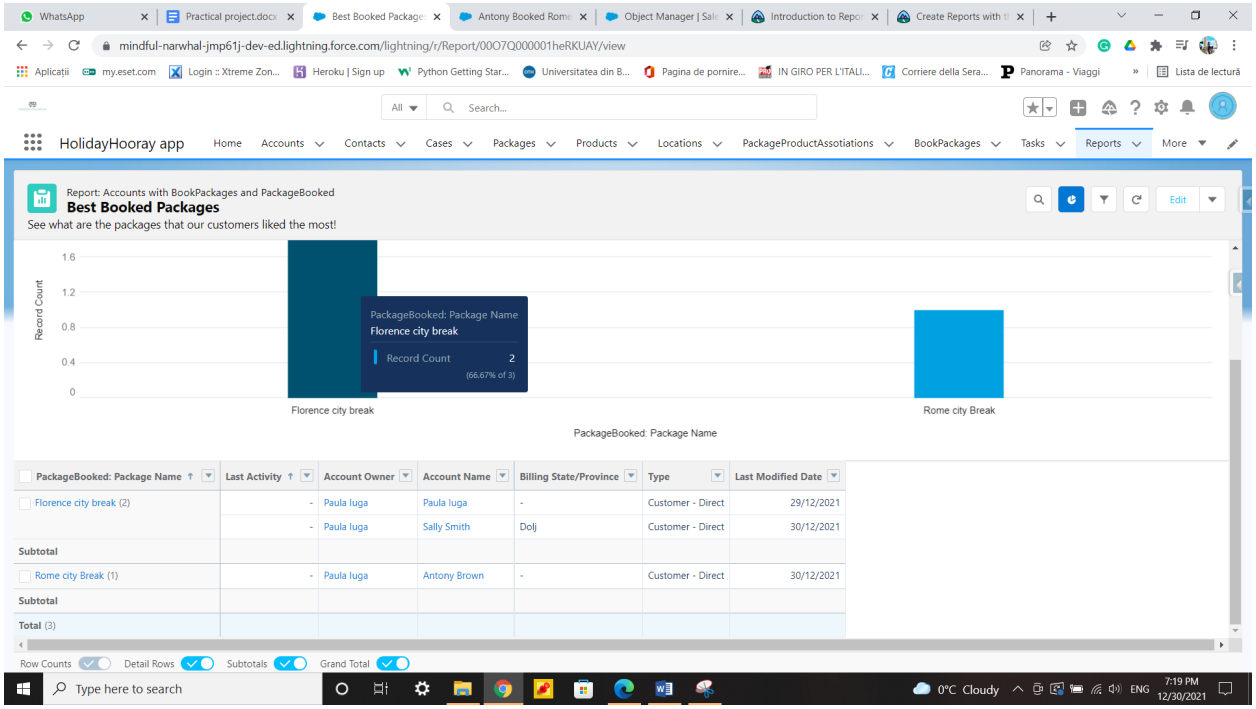
Theme colors are selected from our Brand icon. We updated images for account, background and we changed loading Page logo to our logo.



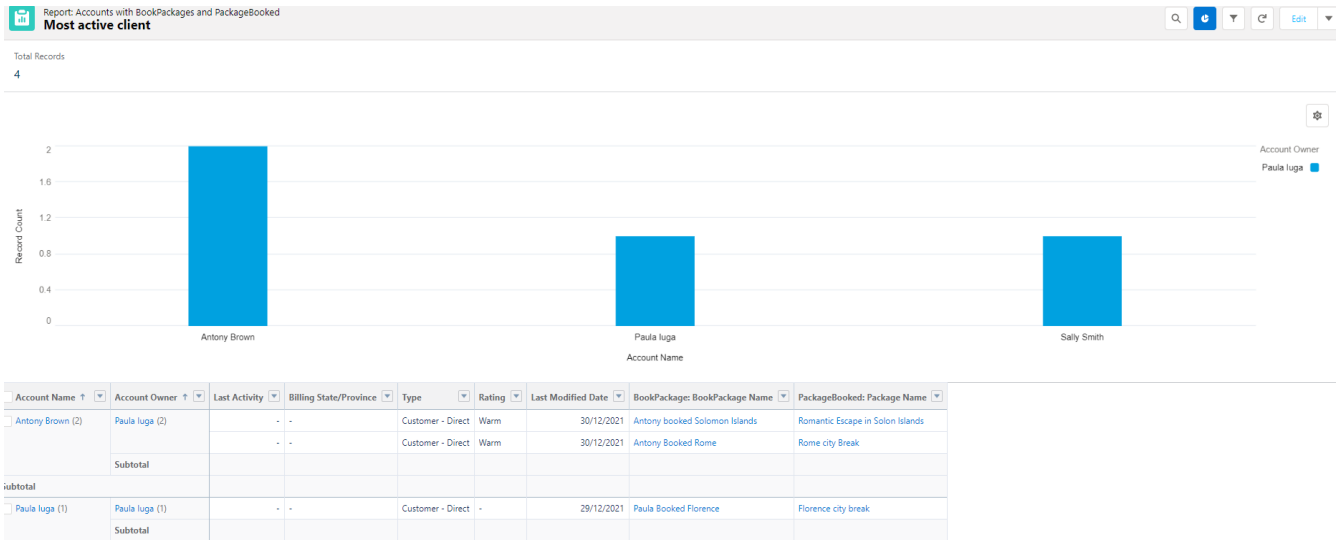


Reports & Dasboards:

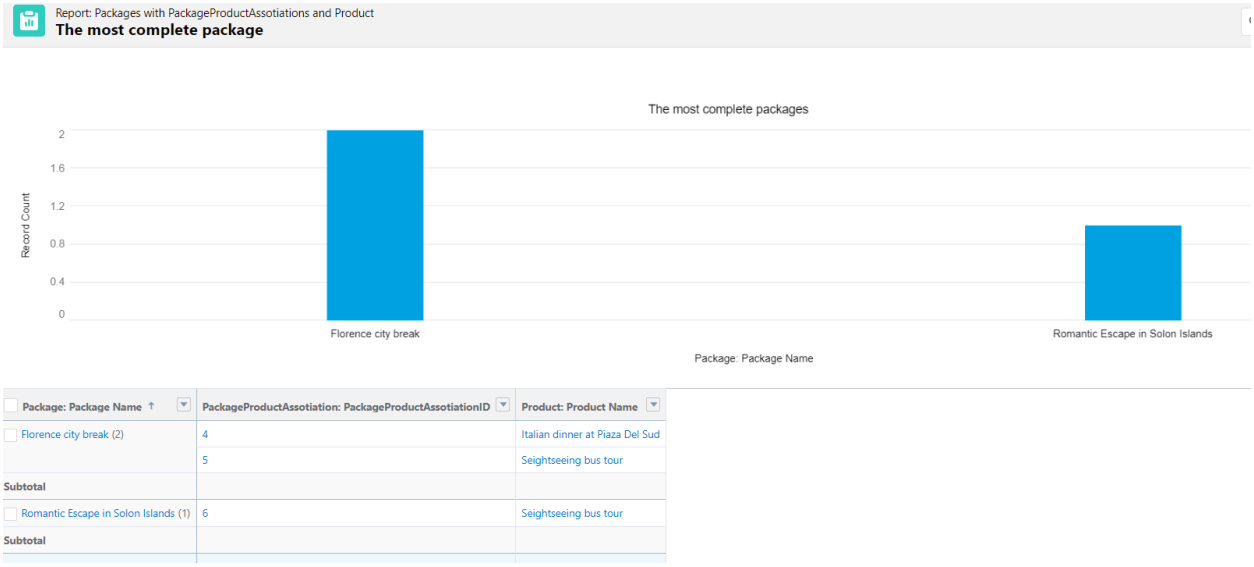
- Report to see what are the most booked packages:



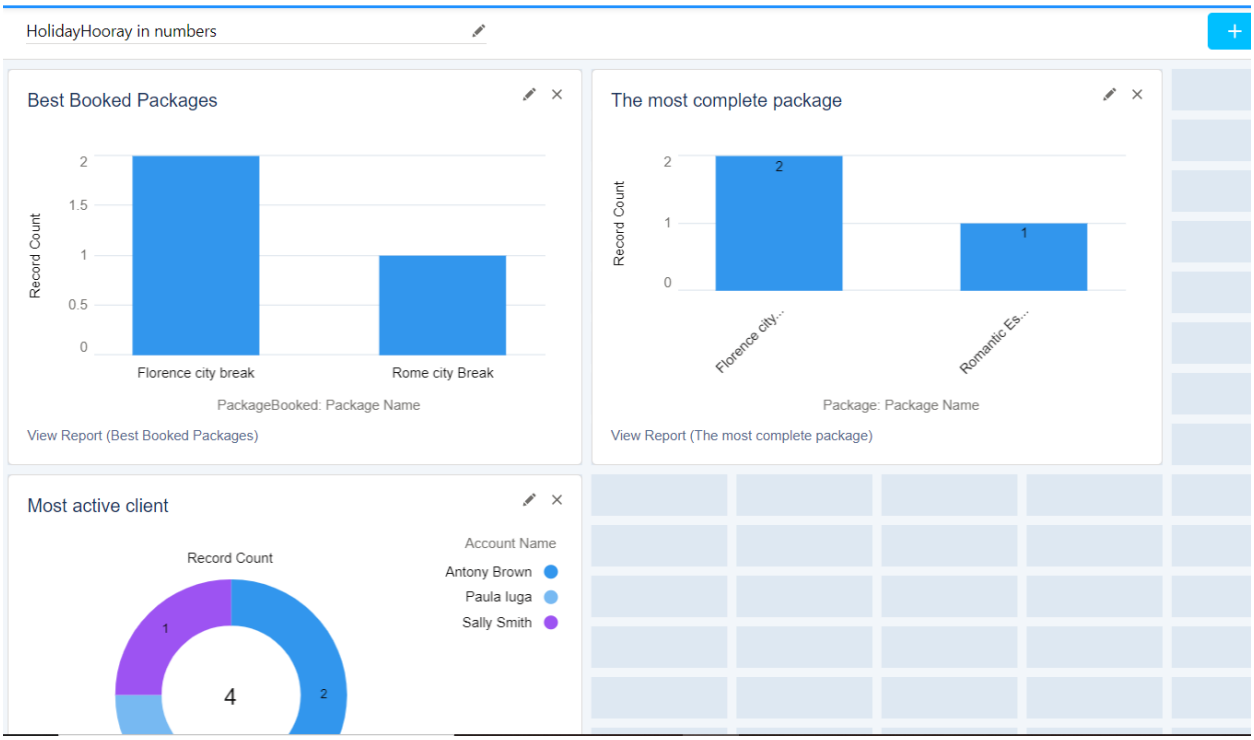
- Most active client:



- The most complete packages:



- Dashboard:



Tests

In order to enforce best practices and ensure code quality, we've created test classes for the custom code automations.

- **Data Factory**
  In order to keep things clean, we used a Data Factory class to generate the records needed for the tests. Since all methods are similar, will attach screenshot showing only the first one. Should be noted that the class itself is marked as @isTest because we use it to create the tests themselves, so it doesn't need tests. Also, it helps to bypass some Salesforce limitations.

```
@isTest
public class HH_DataFactory {
    //We need a valid email to test notifications

    public static List<Account> createAccounts(Integer nrAccs, Boolean executeDML, String name, String email) {
        List<Account> toInsert = new List<Account>();
        for(Integer i = 0; i< nrAccs; i++)
        {
            Account acc = new Account();
            acc.Name = name + i;
            acc.Email__c = email;
            toInsert.add(acc);
        }
        if(executeDML)
            insert toInsert;

        return toInsert;
    }
```

- **HttpCalloutMock**
  Because a QR code is generated for each BookRecord inserted in the database, we have to use an HttpCalloutMock class to simulate the actual API responses.

```
@isTest
global class HH_MockQRCodeResponse implements HttpCalloutMock {

    global HttpResponse respond(HttpRequest req) {
        HttpResponse res = new HttpResponse();
        res.setHeader('Content-Type', 'image/png');
        res.setBody('PNG QR');
        res.setStatusCode(200);

        return res;
    }

}
```

- **The Actual Tests**

  First, we create the data using the DataFactory class

```
@isTest
public class HH_BookPackageTriggerTest {
    private static final Integer NUMBER_BOOKPACKAGES = 3;

    @testSetup
    static void createTestData()
    {
        Test.setMock(HttpCalloutMock.class, new HH_MockQRCodeResponse()); //Need to mock the QR response for the creation of bookPackages
        List<Package__c> testPackages = HH_DataFactory.createPackages(1, true, 'TestPackageFactory', 'Family', 'Florence');
        List<Account> testAccounts = HH_DataFactory.createAccounts(1, true, 'TestAcountFactory', 'radu.ndlcu@gmail.com');
        List<BookPackage__c> testBookPackages = HH_DataFactory.createBookPackages(NUMBER_BOOKPACKAGES, true, 'TestBookPackageFactory', testAccounts[0].ID, testPackages[0].ID);
    }
```

Then create the use cases and use the System.assert to check if everything works properly.

```
@isTest
public static void shouldReceiveSuccessResponseWhenCallingQRGeneratorAPI()
{
    Test.setMock(HttpCalloutMock.class, new HH_MockQRCodeResponse());

    Test.startTest();

    HttpResponse res = BookPackageTriggerHandler.sendRequest(new HttpRequest());

    Test.stopTest();

    System.assert(res.getHeader('Content-Type') == 'image/png');
    System.assert(res.getBody() == 'PNG QR');
    System.assert(res.getStatusCode() == 200);
}

@isTest
public static void shouldPopulateCustomerEmailField()
{
    List<BookPackage__c> testBookPackages = [SELECT CustomerEmail__c FROM BookPackage__c];
    for(BookPackage__c bp : testBookPackages)
        System.assert(bp.CustomerEmail__c != '', 'Customer Email field shouldn\'t be empty');
}

@isTest
public static void shouldInsertAttachmentsAndPopulateQRCodeField(){
    List<BookPackage__c> testBookPackages = [SELECT ID,QRCode__c FROM BookPackage__c];
    for(BookPackage__c bp : testBookPackages)
        System.assert(bp.QRCode__c != '', 'QR Code field shouldn\'t be empty');

    List<Attachment> createdAttachments = [SELECT ID, parentID FROM Attachment];
    System.assert(createdAttachments.size() == NUMBER_BOOKPACKAGES, 'Each bookPackage should have its QR Code');

    Set<Id> ids = (new Map<Id, BookPackage__c>(testBookPackages)).keySet(); //Transform the list of BookPackages into a Set of IDs
    for(Attachment attach : createdAttachments)
        System.assert(ids.contains(attach.parentID), 'All created attachments should have the parentID set to a new BookPackage');
}
```

```
@isTest
private class HH_PaymentTriggerTest {

    private static final string BEFORE_UPDATE_ERROR = 'Completed payments cannot be edited';

    @testSetup
    static void createTestData()
    {
        Test.setMock(HttpCalloutMock.class, new HH_MockQRCodeResponse()); //Need to mock the QR response for the creation of bookPackages
        List<Package__c> testPackages = HH_DataFactory.createPackages(1, true, 'TestPackageFactory', 'Family', 'Florence');
        List<Account> testAccounts = HH_DataFactory.createAccounts(1, true, 'TestAcountFactory', 'radu.ndlcu@gmail.com');
        List<BookPackage__c> testBookPackages = HH_DataFactory.createBookPackages(1, true, 'TestBookPackageFactory', testAccounts[0].ID, testPackages[0].ID);
        List<Payment__c> testPayments = HH_DataFactory.createPayments(1, true, 'TestPaymentFactory', testBookPackages[0].ID, 'Pending', 300);
        List<Payment__c> testPaymentsCompleted = HH_DataFactory.createPayments(1, true, 'TestPaymentFactory', testBookPackages[0].ID, 'Completed', 100);
    }

    @isTest
    public static void shouldThrowErrorWhenUpdatedPaymentIsAlreadyCompleted()
    {
        Payment__c pay = [SELECT ID FROM Payment__c WHERE Status__c = 'Completed' LIMIT 1];
        pay.Amount__c = 2424;

        try {
            Test.startTest();
            update pay;
            Test.stopTest();
        } catch(Exception e) { System.assert(e.getMessage().contains(BEFORE_UPDATE_ERROR), 'Error was '+ e.getMessage()); }
    }

    @isTest
    public static void shouldGenerateInvoicesWhenPaymentIsCompleted()
    {
        Payment__c pay = [SELECT ID FROM Payment__c WHERE Status__c = 'Pending' LIMIT 1];
        pay.Status__c = 'Completed';

        Test.startTest();
        update pay;
        Test.stopTest();

        List<Invoice__c> createdInvoices = [SELECT ID FROM Invoice__c];
        System.debug(createdInvoices.size());
        System.assert(createdInvoices.size() == 1);
```